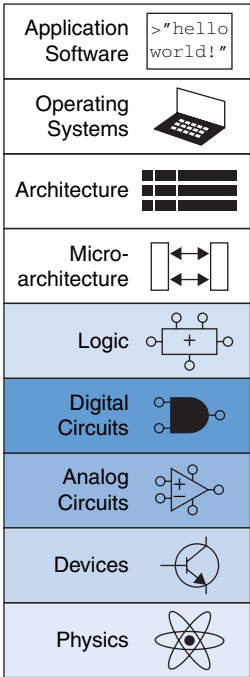


1.1 游戏计划

微处理器在过去三十年间彻底改变了我们的世界。如今一台笔记本电脑的性能远超昔日房间大小的主机。一辆豪华汽车中大约包含100个微处理器。微处理器的进步使手机和互联网成为可能，极大改善了医疗水平，并改变了战争的方式。全球半导体行业销售额从1985年的210亿美元增长到2020年的4000亿美元，而微处理器是这些销售额的主要组成部分。我们相信，微处理器不仅在技术、经济和社会层面具有重要意义，而且本身也是一项引人入胜的人类发明。当你读完这本书时，你将知道如何设计和构建自己的微处理器。在此过程中学到的技能将为你设计许多其他数字系统做好准备。

我们假设您对电学有基本了解，具备一定的编程经验，并且对理解计算机底层运作机制有真正的兴趣。本书聚焦于数字系统的设计，这类系统以1和0为基础运行。我们从数字逻辑门开始，这些逻辑门以1和0作为输入，并输出1和0。接着，我们将探讨如何将逻辑门组合成更复杂的模块，例如加法器和存储器。然后，我们将转向汇编语言编程——微处理器的原生语言。最后，我们将把逻辑门整合起来，构建一个能够运行这些汇编语言程序的微处理器。

数字系统的一大优势在于其基本构件非常简单：只有1和0。它们不需要繁琐的数学或深奥的物理知识。相反，设计者的挑战在于将这些简单的模块组合成复杂的系统。微处理器可能是你构建的第一个复杂到无法容纳于



一下子全部处理。贯穿本书的一个主要主题是如何管理复杂性。

1.2 管理复杂性的艺术

工程师或计算机科学家与外行人的区别之一在于其系统化的复杂性管理方法。现代数字系统由数百万或数十亿个晶体管构建而成。没有人能够通过描述每个晶体管中电子运动的方程并同时求解所有方程来理解这些系统。你需要学会管理复杂性，才能理解如何构建微处理器，而不陷入细节的泥沼。

1. 2. 1 抽象

管理复杂性的关键技巧是*abstraction*：在不重要的细节上将其隐藏。系统可以从多个不同的抽象层次来观察。例如，美国政治家将世界抽象为城市、县、州和国家。一个县包含多个城市，一个州包含多个县。当一位政治家竞选总统时，他主要关心整个州如何投票，而不是每个县如何投票，因此州是最有用的抽象层次。另一方面，人口普查局测量每个城市的人口，因此该机构必须考虑更低抽象层次的细节。

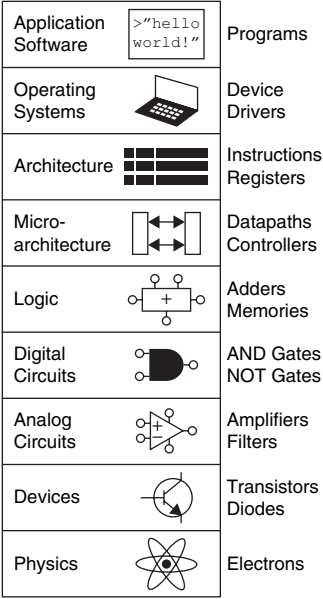


图1.1 电子计算系统的抽象层次

图1.1展示了电子计算机系统的抽象层次，以及每个层次上的典型构建模块。最低抽象层次是物理学，即电子的运动。电子的行为由量子力学和麦克斯韦方程组描述。我们的系统由诸如晶体管（或曾经使用的真空管）之类的电子*devices*构成。这些器件具有称为*terminals*的明确定义的连接点，并且可以通过在每个端子上测量的电压与电流之间的关系来建模。通过抽象到这一器件层次，我们可以忽略单个电子。下一个抽象层次是*analog circuits*，其中器件被组装成诸如放大器之类的组件。模拟电路输入和输出连续范围的电压。*Digital circuits*（例如逻辑门）将电压限制在离散范围内，我们将用这些范围来表示0和1。在逻辑设计中，我们从数字电路构建更复杂的结构，例如加法器或存储器。

*Microarchitecture*连接了抽象的逻辑层和架构层。*architecture*抽象层从程序员的角度描述了计算机。例如，Intel x86架构

大多数微处理器使用的 *personal computers* (PCs) 由一组指令和寄存器（用于临时存储变量的内存）定义，程序员可以自由使用这些指令和寄存器。微架构涉及组合逻辑元件来执行架构所定义的指令。一种特定的架构可以通过多种不同的微架构实现，这些微架构在价格/性能/功耗之间有不同的权衡。例如，Intel Core i7、Intel 80486 和 AMD Athlon 都通过不同的微架构实现了 x86 架构。

进入软件领域，操作系统负责处理底层细节，例如访问硬盘或管理内存。最终，应用软件利用操作系统提供的这些功能为用户解决问题。得益于抽象的强大能力，您的祖母可以毫无顾虑地浏览网页，无需关心电子的量子振动或计算机内存的组织方式。

本书聚焦于从数字电路到计算机架构的抽象层级。当你在某一抽象层级工作时，了解其上下相邻层级的特性将大有裨益。例如，计算机科学家若不理解程序所针对的架构，便无法充分优化代码；器件工程师若不了解晶体管将用于何种电路，便无法在晶体管设计中做出明智的权衡。我们希望，当你读完本书时，能够选择适合解决当前问题的抽象层级，并评估设计选择对其他抽象层级的影响。

1.2.2 纪律

Discipline 是有意限制设计选择的行为，以便能够在更高的抽象层次上更高效地工作。使用可互换零件是这一原则的常见应用。早期著名的可互换零件案例之一出现在汽车制造业。尽管现代汽油动力汽车可追溯至 1886 年德国奔驰专利汽车，但早期汽车是由熟练工匠手工打造的，这一过程既耗时又昂贵。亨利·福特在 1908 年取得了关键性突破，他专注于采用可互换零件和移动装配线进行大规模生产。

可互换零件的规范彻底改变了工业。通过将零部件限制在具有明确公差的标准集合内，汽车的组装和维修速度大大加快，且所需技能更低。汽车制造商不再需要关注较低层次的抽象问题，例如将车门安装到非标准化的开口上。

本书每一章的开头都有一个抽象图标，以深蓝色标示该章的重点，次要主题则以浅蓝色显示。

福特以一份大胆的宣传推出了 T 型车：

我要为大众制造一款汽车。它要足够大，能容纳一家人，又要足够小，让个人也能轻松驾驶和保养。它将采用最优质的材料，由最优秀的工匠，按照现代工程所能设计出的最简洁的构造来打造。但它的价格会非常低廉，任何一份体面薪水的人都能买得起——从而与家人在上帝广阔的天地间，享受数小时的愉悦时光。[《我的生活与工作》，塞缪尔·克劳瑟与亨利·福特著，1922年]



福特T型车照片来自 https://commons.wikimedia.org/wiki/File:TModel_launch_Geelong.jpg

福特T型车成为其时代产量最高的汽车，销量超过1500万辆。纪律性的另一个例证是福特那句名言：“任何顾客都可以选择他想要的颜色来涂装汽车，只要它是黑色的。”

在本书的背景下，数字电路将非常重要。数字电路使用离散电压，而模拟电路使用连续电压。因此，数字电路是模拟电路的一个子集，并且在某种意义上，其能力必然不及更广泛的模拟电路类别。然而，数字电路的设计要简单得多。通过将自身限制在数字电路领域，我们可以轻松地将组件组合成复杂的系统，这些系统在许多应用中最终超越了由模拟组件构建的系统。例如，数字电视、光盘（CD）和手机正在取代它们的模拟前身。

1.2.3 三个Y

除了抽象和规范，设计师还使用三个“-性”来管理复杂性：层次性、模块性和规律性。这些原则适用于软件和硬件系统。

Hierarchy 涉及将系统划分为模块，然后进一步细分每个模块，直到各部分易于理解。► *Modularity* 指出模块具有明确的功能和接口，以便它们能够轻松连接，而不会产生未预料到的副作用。► *Regularity* 追求模块之间的统一性。通用模块被多次复用，从而减少了必须设计的独特模块数量。

为了说明这些“-y”，我们回到汽车制造的例子。在20世纪初，汽车是日常使用中最复杂的物品之一。利用层级原则，我们可以将福特T型车分解为多个组件，例如底盘、发动机和座椅。而发动机又包含四个气缸、一个化油器、一个磁电机和一个冷却系统。化油器则包含燃油和空气进气口、阻风门和节气门、针阀等，如图1.2所示。燃油进气口由螺纹弯头和连接螺母制成。因此，这个复杂系统被递归地分解为可大规模生产的简单可互换组件。

模块化原则教导我们，每个组件都应具有明确的功能和接口。例如，连接螺母的功能是将燃油供给管固定到进气弯管上，且方式应确保不会

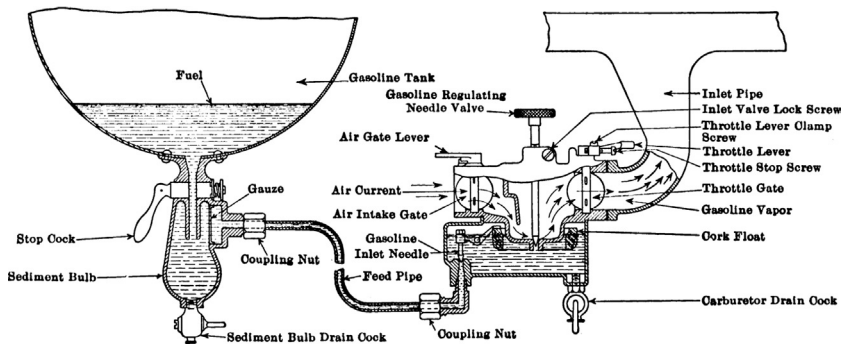


图1.2 T型车燃油系统剖视图，左侧为燃油供给装置，右侧为化油器。（https://en.wikipedia.org/wiki/Ford_Model_T_engine#/media/File:Pagé_1917_Model_T_Ford_Car_Figure_14.png）

泄漏仍可在需要更换进料管时轻松移除。其直径和螺纹螺距均为标准化规格，使用标准化扳手按标准扭矩拧紧。只要指定了正确尺寸，汽车制造商可从多家不同供应商处采购该螺母。模块化原则要求不得产生副作用：连接螺母不应使弯头变形，且最好位于无需拆卸发动机其他设备即可拧紧或移除的位置。

规律性表明，可互换零件是个好主意。有了规律性，漏油的化油器可以用相同的零件替换。化油器可以在装配线上高效生产，而无需费时费力地手工制作。

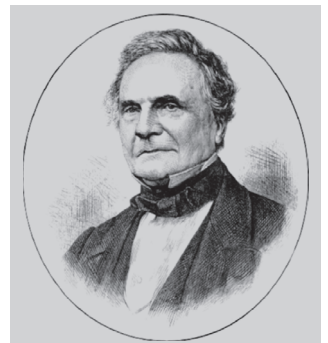
在本书中，我们将反复回到这些关于层次性、模块化和规律性的原则。

1.3 数字抽象

大多数物理变量是连续的。例如，导线上的电压、振荡的频率或质量的位置都是连续量。而数字系统则用 *discrete-valued variables* 表示信息——即具有有限个不同值的变量。

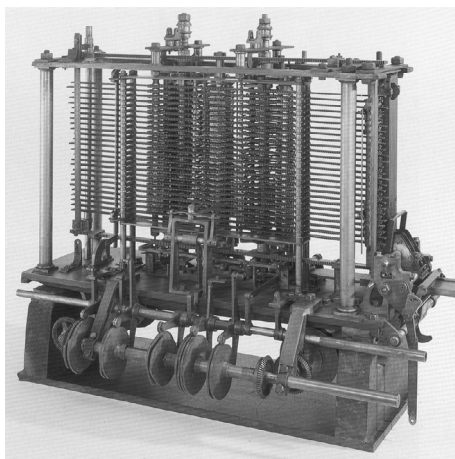
采用具有十个离散值变量的早期数字系统是查尔斯·巴贝奇的分析机。巴贝奇从1834年到1871年辛勤工作，设计并尝试建造这台机械计算机。分析机使用了标有0到9十个位置的齿轮，很像汽车中的机械里程表。图1.3展示了分析机的原型，其中每一行处理一个数字。巴贝奇选择了25行齿轮，因此该机器具有25位精度。

与巴贝奇的机器不同，大多数电子计算机采用二进制（二值）表示法，其中高电压表示“1”，而



查尔斯·巴贝奇（1791–1871）曾就读于剑桥大学，并于1814年与乔治亚娜·惠特莫尔结婚。他发明了分析机，这是世界上第一台机械计算机。他还发明了排障器和统一邮资制度。他对开锁感兴趣，但厌恶街头音乐家（图片由瑞士Fourmilab提供，网址：www.fourmilab.ch）。

图1.3 巴贝奇的分析机，在他1871年去世时仍在建造中（图片由科学博物馆/科学与社会图片库提供）



低电压表示“0”，因为区分两个电压比区分十个电压更容易。

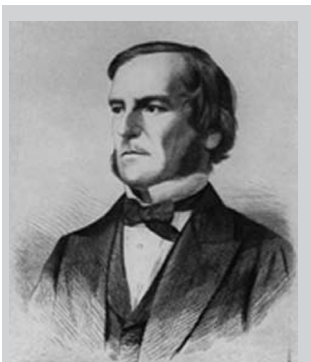
在具有 N 个不同状态的离散值变量中，*amount of information* D 以 *bits* 为单位进行测量，即

$$D = \log_2 N \text{ bits} \quad (1.1)$$

一个二进制变量传递 $\log_2 2 = 1$ 比特的信息。实际上，单词 *bit* 是 *binary digit* 的缩写。巴贝奇齿轮的每个齿位承载 $\log_2 10 = 3.322$ 比特的信息，因为它可以处于 $2^{3.322} = 10$ 种独特位置之一。理论上，连续信号包含无限量的信息，因为它可以取无限多个值。实际上，对于大多数连续信号，噪声和测量误差将信息限制在仅 10 到 16 比特。如果测量必须快速进行，信息含量会更低（例如 8 比特）。

本书聚焦于使用二进制变量（1和0）的数字电路。乔治·布尔开发了一套基于二进制变量的逻辑运算系统，即如今所称的 *Boolean logic*。布尔的每个变量可以是真或假。电子计算机通常使用正电压表示“1”，零电压表示“0”。在本书中，我们将同义使用术语“1”、“真”和“高”。同样，“0”、“假”和“低”也将互换使用。

digital abstraction 的美妙之处在于，数字设计师可以专注于1和0，而无需关心布尔变量在物理上是用特定电压、旋转齿轮还是液压油位来表示的。计算机程序员可以在不了解计算机硬件细节的情况下工作。另一方面，理解硬件细节能让程序员更好地针对特定计算机优化软件。



乔治·布尔，1815–1864年。出身于工薪阶层家庭，无力承担正规教育费用，布尔自学数学，后成为爱尔兰女王学院的教员。他撰写了《

An Investigation of the Laws of Thought》，其中引入了二进制变量及三种基本逻辑运算：与、或、非（图片由美国物理学会提供）。

单个比特携带的信息量并不多。在下一节中，我们将探讨如何使用比特组来表示数字。在后续章节中，我们还将使用比特组来表示字母和程序。

1.4 数制

你已习惯使用十进制数。在由1和0组成的数字系统中，二进制或十六进制数通常更为便捷。本节将介绍本书后续内容中会使用的各种数制。

1.4.1 十进制数

在小学，你学会了用*decimal*计数和做算术。就像你（很可能）有十根手指一样，十进制数字有十个：0、1、2、……、9。十进制数字组合在一起形成更长的十进制数。十进制数的每一列的权重是前一列的十倍。从右到左，列权重分别为1、10、100、1000，以此类推。十进制数被称为*base 10*。基数用数字后的下标表示，以避免在多种基数下工作时产生混淆。例如，图1.4展示了十进制数9742₁₀如何写成其每一位数字乘以对应列权重的和。

一个N位十进制数表示10^N种可能性之一：0, 1, 2, 3, …, 10^N - 1。这被称为该数的*range*。例如，一个三位十进制数表示0到999范围内1000种可能性之一。

1.4.2 二进制数

位表示两个值之一，0或1，它们组合在一起形成*binary numbers*。二进制数的每一列的权重是前一列的两倍，因此二进制数是*base 2*。在二进制中，列权重（同样从右到左）为1、2、4、8、16、32、64、128、

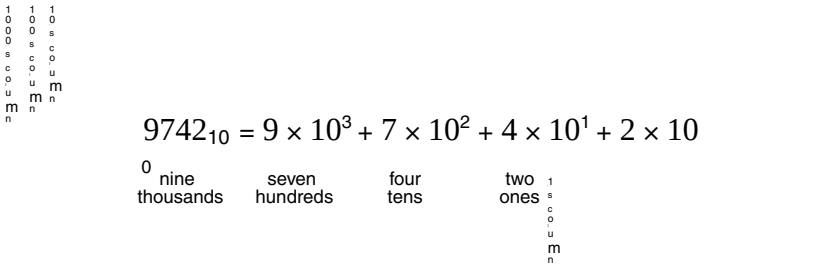


图1.4 十进制数的表示

256、512、1024、2048、4096、8192、16384、32768、65536，以此类推。如果你经常处理二进制数，记住这些2的幂次直到 2^{16} 会节省时间。

一个 N 位的二进制数表示 2^N 种可能性之一： $0, 1, 2, 3, \dots, 2^N - 1$ 。表 1.1 展示了 1 位、2 位、3 位和 4 位二进制数及其对应的十进制数。

Example 1.1 BINARY TO DECIMAL CONVERSION

Convert the binary number 10110_2 to decimal.

Solution Figure 1.5 shows the conversion.

表1.1 二进制数及其十进制等值

1-Bit Binary Numbers	2-Bit Binary Numbers	3-Bit Binary Numbers	4-Bit Binary Numbers	Decimal Equivalents
0	00	000	0000	0
1	01	001	0001	1
	10	010	0010	2
	11	011	0011	3
		100	0100	4
		101	0101	5
		110	0110	6
		111	0111	7
			1000	8
			1001	9
			1010	10
			1011	11
			1100	12
			1101	13
			1110	14
			1111	15

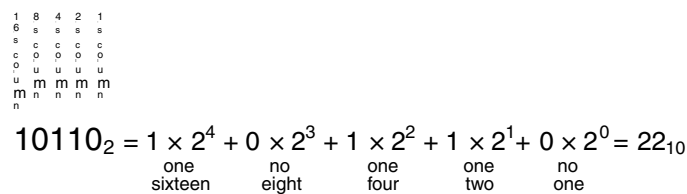


图1.5 二进制数转换为十进制数

示例1.2 十进制到二进制的转换

将十进制数84₁₀转换为二进制。

解决方案 确定二进制结果的每一列是1还是0。我们可以从最左边或最右边的列开始进行此操作。

从左边开始，首先找到小于或等于该数的最大2的幂（本例中为64）。84 ≥ 64，因此在64的列上记1，剩余84 − 64 = 20。20 < 32，因此在32的列上记0。20 ≥ 16，因此在16的列上记1，剩余20 − 16 = 4。4 < 8，因此在8的列上记0。4 ≥ 4，因此在4的列上记1，剩余4 − 4 = 0。因此，在2的列和1的列上必须记0。综合起来，84₁₀ = 1010100₂。

从右侧开始，反复将该数除以2。余数填入每一列。84/2 = 42，因此个位列填入0。42/2 = 21，因此2位列填入0。21/2 = 10，余数1填入4位列。10/2 = 5，因此8位列填入0。5/2 = 2，余数1填入16位列。2/2 = 1，因此32位列填入0。最后，1/2 = 0，余数1填入64位列。同样地，84₁₀ = 1010100₂。

1.4.3 十六进制数

编写长二进制数变得繁琐且容易出错。每四位一组表示2⁴ = 16种可能性之一。因此，有时使用base 16（称为hexadecimal）更为方便。十六进制数使用数字0到9以及字母A到F，如表1.2所示。以16为基数的列权重分别为1、16、16²（或256）、16³（或4096），依此类推。

Hexadecimal，这个术语由IBM于1963年创造，源自希腊语hexi（六）和拉丁语decem（十）。更恰当的术语本应使用拉丁语sexa（六），但sexadecimal听起来过于轻佻。

示例1.3 十六进制到二进制和十进制的转换

将十六进制数2ED₁₆转换为二进制和十进制。

十六进制与二进制之间的转换很简单，因为每个十六进制数字直接对应四位二进制数字。2₁₆ = 0010₂，E₁₆ = 1110₂，D₁₆ = 1101₂，因此2ED₁₆ = 001011101101₂。转换为十进制则需要如图1.6所示的算术运算。

表1.2 十六进制数字系统

Hexadecimal Digit	Decimal Equivalent	Binary Equivalent
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

图1.6 十六进制数转换为十进制数

2
5
6
s
c
o
u
n
t
m
n

1
6
s
c
o
u
n
t
m
n

1
6
s
c
o
u
n
t
m
n

$$2ED_{16} = 2 \times 16^2 + E \times 16^1 + D \times 16^0 = 749_{10}$$

two
two hundred
fifty six's

fourteen
sixteens

thirteen
ones

示例1.4 二进制到十六进制转换

将二进制数1111010₂转换为十六进制。

解法同样简单。从右侧开始读取。最低四位是1010₂ = A₁₆。接下来的位是111₂ = 7₁₆。因此，1111010₂ = 7A₁₆。

示例1.5 十进制到十六进制和二进制的转换

将十进制数 333_{10} 转换为十六进制和二进制。

与十进制转二进制类似，十进制转十六进制可以从左或从右进行。

从左边开始，取小于或等于该数（本例中为256）的最大16的幂。256乘以333得1次，因此在256的列上记1，剩余 $333 - 256 = 77$ 。16乘以77得4次，因此在16的列上记4，剩余 $77 - 16 \times 4 = 13$ 。 $13_{10} = D_{16}$ ，因此在1的列上记D。总结： $333_{10} = 14D_{16}$ 。现在，如例1.3所示，从十六进制转换为二进制就很容易了。 $14D_{16} = 1001101101_2$ 。

从右边开始，反复将该数除以16。余数填入每一列。 $333/16 = 20$ ，余数 $13_{10} = D_{16}$ 填入个位列。 $20/16 = 1$ ，余数4填入16位列。 $1/16 = 0$ ，余数1填入256位列。结果同样是 $14D_{16}$ 。

1.4.4 字节、半字节及其他相关概念

一组八位称为一个 *byte*。它表示 2^8 种=256种可能性中的一种。计算机存储器中存储的对象的大小通常以字节而非位来衡量。

四位一组，即半个字节，称为 *nibble*。它表示 $2^4 = 16$ 种可能性之一。一个十六进制数字存储一个半字节，两个十六进制数字存储一个完整字节。半字节已不再是常用的单位，但这个术语颇为有趣。

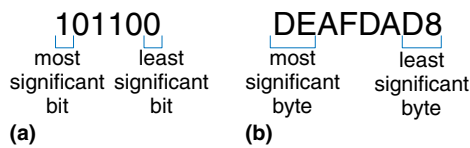
微处理器以称为 *words* 的数据块处理数据。字的大小取决于微处理器的架构。本章撰写于2021年时，大多数计算机采用64位处理器，这意味着它们以64位字运行。当时，处理32位字的旧式计算机也广泛存在。更简单的微处理器，尤其是用于烤面包机等设备中的，则使用8位或16位字。

在一组位中，位于1's列的位称为 *least significant bit (lsb)*，另一端的位称为 *most significant bit (msb)*，如图1.7(a)所示的一个6位二进制数。类似地，在一个字中，字节被标识为 *least significant byte (LSB)* 到 *most significant byte (MSB)*，如图1.7(b)所示的一个用八位十六进制数字表示的4字节数。

巧的是， $2^{10} = 1024 \approx 10^3$ 。因此，术语 *kilo*（希腊语意为千）表示 2^{10} 。例如， 2^{10} 字节等于一

microprocessor 是构建在单个芯片上的处理器。直到20世纪70年代，处理器都过于复杂，无法集成到单个芯片上，因此大型机处理器由包含多个芯片的电路板构建而成。英特尔于1971年推出了第一款4位微处理器，名为4004。如今，即使是最复杂的超级计算机也使用微处理器构建。在本书中，我们将互换使用术语 *microprocessor* 和 *processor*。

图1.7 最低和最高有效位与字节



为了区分十的幂次和二
的幂次，针对二的幂次
出现了一种特定的命名
约定：2¹⁰ 称为千比（Ki）
，2²⁰ 称为兆比（Mi）
，2³⁰ 称为吉比（Gi）
，2⁴⁰ 称为太比（Ti）
，2⁵⁰ 称为拍比（Pi）
，2⁶⁰ 称为艾比（Ei）
，以此类推。例如，2³⁰
字节即为一个吉比字节
（GiB）。

片上内存和RAM以2的
幂次计量，而硬盘则以10
的幂次计量。例如，32
GB（GiB）的RAM实际
上是34,359,738,368字
节的内存，而硬盘上的32
GB则是32,000,000字
节的内存。

千字节（1 KB）。类似地，*mega*（百万）表示2²⁰ ≈ 10⁶，而*giga*（十亿）表示2³⁰ ≈ 10⁹。如果你知道2¹⁰ ≈ 1千，2²⁰ ≈ 1百万，2³⁰ ≈ 1十亿，并记住2⁹以内的2的幂，就能轻松心算任何2的幂的近似值。

示例1.6 估算2的幂

不使用计算器，求2²⁴的近似值。

解法 将指数拆分为十的倍数和余数。

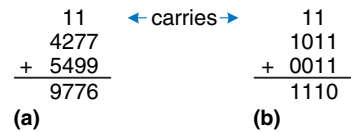
224 = 2²⁰ × 2⁴。2²⁰ ≈ 100万。2⁴ = 16。因此，2²⁴ ≈ 1600万。严格来说，2²⁴ = 16,777,216，但1600万对于营销目的来说已经足够接近了。

1024字节称为*kilobyte*（KB）或*kibibyte*（KiB）。1024比特称为*kilobit*（Kb或Kbit）或*kibibit*（Kib或Kibit）。类似地，MB/MiB、Mb/Mib、GB/GiB、Gb/Gib用于表示百万和十亿级别的字节和比特。内存容量通常以字节为单位衡量。通信速度通常以十的幂次方比特/秒为单位衡量。例如，拨号调制解调器的最大速度通常为56千比特/秒，即56,000比特/秒。

1.4.5 二进制加法

二进制加法与十进制加法非常相似，但更为简单，如图1.8所示。与十进制加法一样，如果两个数之和大于单个数字所能容纳的值，我们就在下一列*carry*一个1。图1.8比较了十进制和二进制数的加法。在图1.8(a)的最右列，7 + 9 = 16，由于大于9，无法放入单个数字。因此，我们记录个位数6，并将十位数1进位到下一列。同样，在二进制中，如果两个数之和大于1，我们便将2的位数进位到下一列。例如，在图1.8(b)的最右列，其和为

图1.8 加法示例显示进位：
a) 十进制 b) 二进制



$1 + 1 = 2_{10} = 10_2$ 无法放入单个二进制位中。因此，我们记录结果的个位数字（0），并将结果的二位数字（1）进位到下一列。在第二列中，总和为 $1 + 1 + 1 = 3_{10} = 11_2$ 。同样，我们记录个位数字（1），并将二位数字（1）进位到下一列。显而易见，被进位到相邻列的位称为 *carry bit*。

示例1.7 二进制加法

计算 $0111_2 + 0101_2$ 。

解 图1.9显示和为 1100_2 。进位以蓝色标出。我们可以通过十进制重复计算来验证结果。 $0111_2 = 7_{10}$ 。 $0101_2 = 5_{10}$ 。和为 $12_{10} = 1100_2$ 。

0111

0101

1

100111

图1.9 二进制加法示例

数字系统通常以固定位数进行运算。当加法结果超出可用位数范围时，称为 *overflow*。例如，4位数的范围为[0, 15]。若4位二进制加法结果超过15，则发生溢出。和的第五位被丢弃，导致剩余四位产生错误结果。通过检查最高位列的进位输出可检测溢出。

示例1.8 带溢出的加法

计算 $1101_2 + 0101_2$ 。是否发生溢出？

解 图1.10显示和为 10010_2 。该结果超出了4位二进制数的表示范围。若必须将其存储为四位，则丢弃最高位，得到错误结果 0010_2 。若使用五位或更多位数的数字进行计算，结果 10010_2 将是正确的。

1101

0101

1

0010

11

1

图1.10 二进制加法溢出示例

1.4.6 有符号二进制数

到目前为止，我们只考虑了表示正数的*unsigned*二进制数。我们通常需要同时表示正数和负数，这就需要一种不同的二进制数系统。存在几种表示*signed*二进制数的方案。其中最广泛使用的两种被称为 *sign/magnitude*和*two's complement*。

符号/幅值数

*Sign/magnitude*数字直观上很吸引人，因为它们符合我们用负号后跟绝对值表示负数的习惯。一个{v*}位的符号/数值表示法使用最高

1996年6月4日发射的价值70亿美元的阿丽亚娜5型火箭，在升空40秒后偏离航线、解体并爆炸。此次失败是由于控制火箭的计算机溢出其16位范围并崩溃所致。

该代码曾在阿丽亚娜4型火箭上经过广泛测试。然而，阿丽亚娜5型火箭的发动机速度更快，为控制计算机产生了更大的数值，从而导致溢出。



照片由ESA/CNES/ARIANESPACE- Service Optique CS6提供。

对一个数的位进行取反的结果称为该数的one's complement。例如，0110111的反码是1001000。之所以称为反码，是因为将这两个数相加时，会得到全1'。例如，0110111 + 1001000 = 1111111。

最高有效位作为符号位，其余 $N - 1$ 位表示数值大小（绝对值）。符号位为0表示正数，符号位为1表示负数。

示例1.9 符号/幅值数

将5和-5写成4位符号/数值数。

解 两个数的量级均为 $5_{10} = 101_2$ 。因此， $5_{10} = 0101_2$ ，而 $-5_{10} = 1101_2$ 。

遗憾的是，普通的二进制加法不适用于符号/数值表示法。例如，对 $-5_{10} + 5_{10}$ 使用普通加法会得到 $1101_2 + 0101_2 = 10010_2$ ，这是无意义的。

一个 N 位符号/幅度数的范围是 $[-2^{N-1} + 1, 2^{N-1} - 1]$ 。符号/幅度数有点特殊，因为同时存在+0和-0。两者都表示零。正如你可能预料的，同一个数有两种不同的表示方式可能会带来麻烦。

二进制补码数

Two's complement数字与无符号二进制数相同，只是最高有效位位置的权重为 -2^{N-1} 而不是 2^{N-1} 。它们克服了符号/数值表示法的缺点：零具有单一表示形式，且普通加法运算有效。

在二进制补码表示中，零写作全零： $00 \cdots 000_2$ 。最大的正数在最高位为0，其余位为1： $01 \cdots 111_2 = 2^{N-1} - 1$ 。最小的负数在最高位为1，其余位为0： $10 \cdots 000_2 = -2^{N-1}$ 。而-1写作全一： $11 \cdots 111_2$ 。

注意，正数的最高有效位为0，负数的最高有效位为1，因此最高有效位可视为符号位。然而，对于补码数和原码数，整体数值的解释方式是不同的。

二进制补码数的符号反转（例如，从+5到-5，或从-17到+17）是通过将该数的所有位取反，然后在最低有效位上加1来实现的。这一过程称为reversing the sign方法。它在寻找负数的表示或确定负数的绝对值时非常有用。

Example 1.10 TWO'S COMPLEMENT REPRESENTATION OF A NEGATIVE NUMBER

Find the representation of -2_{10} as a 4-bit two's complement number.

解法 首先表示数值： $+2_{10} = 0010_2$ 。要得到 -2_{10} ，通过取反位并加1来反转符号。对 0010_2 取反得到 1101_2 。 $1101_2 + 1 = 1110_2$ 。因此， -2_{10} 为 1110_2 。 *sign*

示例1.11 负数的二进制补码数值

求4位二进制补码数0x9（即 1001_2 ）的十进制值。

解 1001_2 的最高位为1，因此它表示负数。要求其绝对值，需通过取反加1来反转符号。将 1001_2 取反得 0110_2 ， 0110_2 加1得 0111_2 ，即 7_{10} 。因此， 1001_2 等于 -7_{10} 。

补码数具有一个显著优势，即加法对正数和负数都能正确运算。请注意，当对N位数进行加法时，第N位（即 $N + 1^{\text{th}}$ 结果位）的进位会被丢弃。

示例1.12 补码加法

使用二进制补码数计算 (a) $-2_{10} + 1_{10}$ 和 (b) $-7_{10} + 7_{10}$ 。

解 (a) $-2_{10} + 1_{10} = 1110_2 + 0001_2 = 1111_2 = -1_{10}$ 。 (b) $-7_{10} + 7_{10} = 1001_2 + 0111_2 = 10000_2$ 。第五位被丢弃，留下正确的4位结果 0000_2 。

减法通过取第二个数的二进制补码，然后相加来执行。

E示例1.13 补码减法

RS

使用4位二进制补码计算(a) $5_{10} - 3_{10}$ 和(b) $3_{10} - 5_{10}$ 。

解 (a) $3_{10} = 0011_2$ 。取其补码得到 $-3_{10} = 1101_2$ 。现在，将 $5_{10} + (-3_{10}) = 0101_2 + 1101_2 = 0010_2 = 2_{10}$ 。注意，最高位的进位被丢弃，因为结果存储在四位中。(b) 取 5_{10} 的补码得到 $-5_{10} = 1011$ 。现在，将 $3_{10} + (-5_{10}) = 0011_2 + 1011_2 = 1110_2 = -2_{10}$ 。

0的二进制补码通过将所有位取反（得到 $11\cdots111_2$ ）再加1得到，结果为全0，忽略最高位产生的进位。因此，零始终用全0表示。与符号/数值系统不同，二进制补码系统没有单独的-0。零被视为正数，因为其符号位为0。

reversing the 的过程在历史上被称为“求二的补码”。然而，我们发现这一术语有时会让人误以为在处理二的补码数时必须“求二的补码”，但事实并非如此！这种反转符号的过程仅在某些情况下用于处理负的二的补码数。

二进制补码得名于这样一个事实：求一个数的 *complement*（负数）是通过从 2^N （以其基数 *two's* 表示）中减去该数得到的——因此称为“补码”。例如，要创建 7 的 4 位补码（即 -7），我们执行： $2^4 - 7 = 10000 - 0111 = (1111 + 1) - 0111 = (1111 - 0111) + 1 = 1000 + 1 = 1001$ 。这就是 -7 的 4 位二进制补码表示。注意，最后一步与取反符号法相同，即反转 0111 的位（得到 1000）再加 1。

与无符号数类似， N 位二进制补码数表示 2^N 种可能值之一。然而，这些值在正数和负数之间分配。例如，一个4位无符号数表示16个值：0到15。一个4位二进制补码数也表示16个值：-8到7。通常， N 位二进制补码数的范围是 $[-2^{N-1}, 2^{N-1} - 1]$ 。负数比正数多一个，这是合理的，因为没有-0。最负的数 $10\cdots000_2 = -2^{N-1}$ 有时被称为*weird number*。它的二进制补码通过取反位（得到 $01\cdots111_2$ ）再加1得到，结果又是 $10\cdots000_2$ ，即这个奇怪的数。因此，这个负数没有对应的正数。

两个 N 位正数或负数相加，若结果大于 $2^{N-1}-1$ 或小于 -2^{N-1} ，则可能发生溢出。正数与负数相加永远不会导致溢出。与无符号数不同，最高有效列的进位并不表示溢出。相反，如果相加的两个数具有相同的符号位，而结果具有相反的符号位，则发生溢出。

示例1.14 补码加法
带溢出

使用4位二进制补码数计算 $4_{10} + 5_{10}$ 。结果是否溢出？

解 $4_{10} + 5_{10} = 0100_2 + 0101_2 = 1001_2 = -7_{10}$ 。结果超出了4位正数二进制补码的范围，产生了错误的负数结果。如果使用五位或更多位进行计算，结果 $01001_2 = 9_{10}$ 将是正确的。

当二进制补码数扩展为更多位时，必须将符号位复制到最高有效位位置。这一过程称为*sign extension*。例如，数字3和-3分别表示为4位二进制补码数0011和1101。通过将符号位复制到三个新增的高位，它们被符号扩展为七位，分别形成0000011和1111101，即3和-3的7位表示。

数系比较

最常用的三种二进制数制是无符号数、二进制补码和符号/数值表示法。表1.3比较了这三种数制下 N 位数的范围。二进制补码很方便，因为它既能表示正整数也能表示负整数，并且普通加法适用于所有数字。减法通过对第二个数取负（即反转）来实现。

表1.3 N位数的范围

System	Range
Unsigned	$[0, 2^N - 1]$
Two's Complement	$[-2^{N-1}, 2^{N-1} - 1]$
Sign/Magnitude	$[-2^{N-1} + 1, 2^{N-1} - 1]$

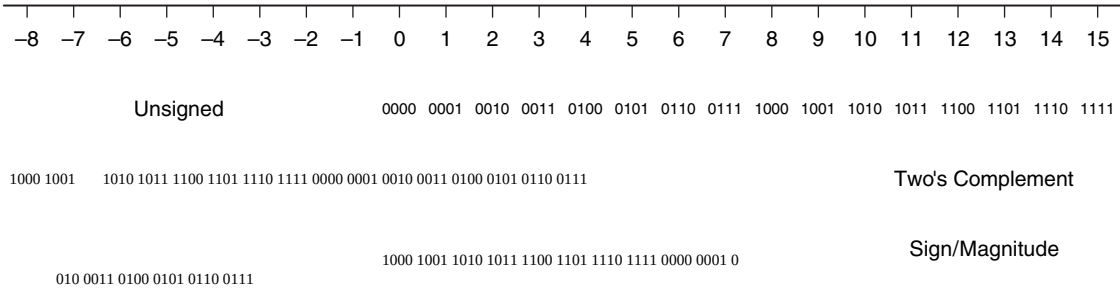


图1.11 数轴与4位二进制编码

符号），然后相加。除非另有说明，否则假设所有有符号二进制数均使用二进制补码表示。

图1.11展示了一条数轴，标明了各系统中4位数的值。无符号数按常规二进制顺序覆盖范围[0, 15]。补码数覆盖范围[-8, 7]。非负数[0, 7]与无符号数采用相同编码。负数[-8, -1]的编码方式使得更大的无符号二进制值代表更接近0的数。注意，特殊数1000表示-8，且没有对应的正数。原码数覆盖范围[-7, 7]。最高位为符号位。正数[1, 7]与无符号数采用相同编码。负数对称分布，但符号位被置1。0由0000和1000两种形式表示。因此，由于0有两种表示，N位原码数仅能表示 2^N-1 个整数。N位无符号二进制数或补码数能表示 2^N 个整数。

1.5 逻辑门

既然我们已经知道如何使用二进制变量来表示信息，接下来将探讨对这些二进制变量进行运算的数字系统。Logic gates是简单的数字电路，它们接收一个或多个二进制输入并产生一个二进制输出。逻辑门用符号表示，显示输入（或多个输入）和输出。输入通常绘制为

左侧（或顶部）为输入，右侧（或底部）为输出。数字设计者通常使用字母表前部的字母表示门输入，用字母Y表示门输出。输入与输出之间的关系可以通过真值表或布尔方程来描述。*truth table*在左侧列出输入，在右侧列出对应的输出，每种可能的输入组合对应一行。*Boolean equation*是使用二进制变量的数学表达式。

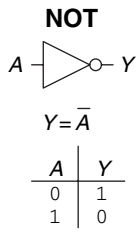


图1.12 非门

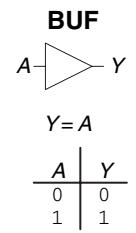


图1.13 缓冲器

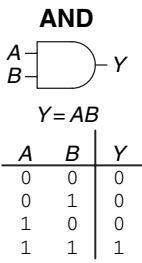


图1.14 与门

根据Perl编程语言发明者拉里·沃尔的说法，“程序员的三大美德是懒惰、不耐烦和傲慢。”

1.5.1 非门

一个NOT gate有一个输入A和一个输出Y，如图1.12所示。非门的输出是其输入的逆。如果A为假，则Y为真。如果A为真，则Y为假。这种关系由图中的真值表和布尔方程总结。布尔方程中A上方的横线读作NOT，因此 $Y = \bar{A}$ 被解读为“Y等于非A”。非门也被称为*inverter*。

其他文本使用多种表示法来表示NOT，包括 $Y = A'$ 、 $Y = \neg A$ 、 $Y = !A$ 或 $Y = \sim A$ 。我们将专门使用 $Y = \bar{A}$ ，但如果你在其他地方遇到其他表示法，不必感到困惑。

1.5.2 缓冲区

另一种单输入逻辑门称为*buffer*，如图1.13所示。它简单地将输入复制到输出。

从逻辑角度来看，缓冲器与导线并无区别，因此看似无用。然而，从模拟角度来看，缓冲器可能具有理想特性，例如能够向电机提供大电流，或快速将其输出发送至多个门电路。这正说明了为何我们需要考虑多个抽象层次才能完全理解一个系统；数字抽象隐藏了缓冲器的真实用途。

三角形符号表示缓冲器。输出端的圆圈称为*bubble*，表示反相，如图1.12中非门符号所示。

1.5.3 与门

双输入逻辑门更有趣。图1.14所示的AND gate仅在A和B均为TRUE时输出TRUE（即Y），否则输出FALSE。按照惯例，输入按00、01、10、11的顺序列出，如同二进制计数。与门的布尔方程有多种写法： $Y = A$

• B ， $Y = AB$ ，或 $Y = A \cap B$ 。 $\cap$ 符号读作“交集”，是逻辑学家偏好的用法。我们更倾向于 $Y = AB$ ，读作“Y 等于 A 且 B、”，因为我们比较懒。

1.5.4 或门

图1.15所示的OR gate在A或B（或两者）为TRUE时产生TRUE输出Y。或门的布尔方程写作 $Y = A + B$ 或 $Y = A \cup B$ 。 $\cup$ 符号读作并集，是逻辑学家偏好的表示法。数字设计者通常使用+记法， $Y = A + B$ 读作“Y等于A或B”。

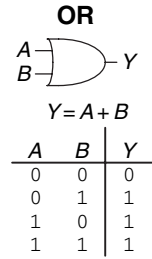


图1.15 或门

1.5.5 其他双输入门

图1.16展示了其他常见的双输入逻辑门。XOR（异或，读作“ex-OR”）在A或B为真但不同时为真时输出为真。异或运算用 $\oplus$ 表示，即一个带圆圈的加号。任何门都可以后接一个气泡来反转其操作。NAND gate执行与非运算，除非两个输入均为真，否则输出为真。NOR gate执行或非运算，若A和B均不为真，则输出为真。一个N输入的异或门有时被称为parity门，当奇数个输入为真时输出为真。与双输入门一样，真值表中的输入组合按计数顺序列出。

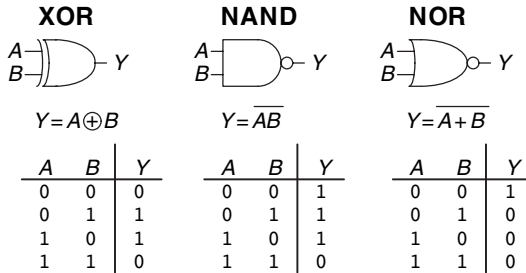


图1.16 更多二输入逻辑门

示例1.15 同或门

图1.17展示了一个两输入XNOR（读作异或非）gate的符号和布尔方程，它执行XOR的逆运算。请完成真值表。

解 图1.18展示了真值表。当两个输入均为FALSE或均为TRUE时，XNOR输出为TRUE。两输入XNOR门有时被称为equality门，因为当输入相等时其输出为TRUE。

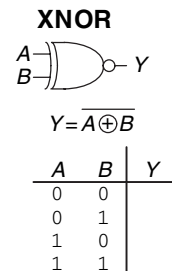
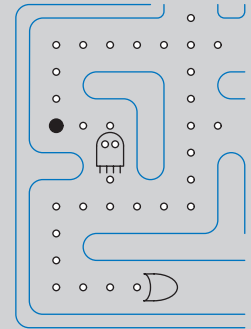


图1.17 XNOR门

1.5.6 多输入门

存在许多具有三个或更多输入的布尔函数。最常见的是AND、OR、XOR、NAND、NOR和XNOR。一个N输入的AND

记住OR符号的一个傻办法是，它的输入侧弯曲得像吃豆人的嘴，所以这个门很饿，愿意吃掉它能找到的任何TRUE输入！



A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

图1.18 XNOR真值表

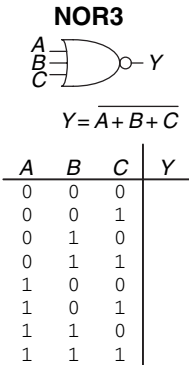


图1.19 三输入或非门

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

图1.20 三输入或非门真值表

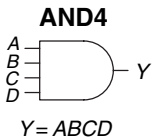


图1.21 四输入与门

当所有N输入均为TRUE时，与门输出TRUE。当至少一个输入为TRUE时，N输入或门输出TRUE。

示例1.16 三输入或非门

图1.19展示了一个三输入或非门的符号和布尔方程。请完成真值表。

解 图1.20展示了真值表。只有当所有输入均为FALSE时，输出才为TRUE。

示例1.17 四输入与门

图1.21展示了一个四输入与门的符号和布尔方程。请创建一个真值表。

解 图1.22展示了真值表。只有当所有输入均为TRUE时，输出才为TRUE。

1.6 数字抽象之下

数字系统使用离散值变量。然而，这些变量由连续的物理量表示，例如导线上的电压、齿轮的位置或气缸中的液位。因此，设计者必须选择一种方法将连续值与离散值关联起来。

例如，考虑用导线上的电压表示二进制信号A。设0伏（V）表示A=0，5伏表示A=1。任何实际系统都必须容忍一定的噪声，因此4.97伏很可能也应被解读为A=1。但4.3伏呢？2.8伏呢？或者2.500000伏呢？

1.6.1 电源电压

假设系统中最低电压为0 V，也称为ground或GND。系统中的最高电压来自电源，通常称为V_{DD}。在1970年代和1980年代的技术中，V_{DD}通常为5 V。随着芯片向更小晶体管发展，V_{DD}已降至3.3 V、2.5 V、1.8 V、1.5 V、1.2 V，甚至更低，以节省功耗并避免晶体管过载。

1.6.2 逻辑电平

连续变量到离散二元变量的映射通过定义logic levels完成，如图1.23所示。第一个门称为driver，第二个门称为receiver。其输出为

驱动器连接到接收器的输入端。驱动器在0到 V_{OL} 范围内产生低电平（0）输出，或在 V_{OH} 到 V_{DD} 范围内产生高电平（1）输出。如果接收器接收到0到 V_{IL} 范围内的输入，它将认为该输入为低电平。如果接收器接收到 V_{IH} 到 V_{DD} 范围内的输入，它将认为该输入为高电平。如果由于噪声或故障元件等原因，接收器的输入落在 V_{IL} 到 V_{IH} 之间的*forbidden zone*范围内，则门电路的行为将不可预测。 V_{OH} 、 V_{OL} 、 V_{IH} 和 V_{IL} 被称为输出和输入的高、低逻辑电平。

1.6.3 噪声容限

若要使驱动器的输出在接收器输入端被正确解读，我们必须选择 V_{OL} 、 $< V_{IL}$ 以及 V_{OH} 、 $> V_{IH}$ 。这样，即使驱动器的输出受到噪声干扰，接收器的输入端仍能检测到正确的逻辑电平。*noise margin*（NM）是指可叠加在最差情况输出上、且信号仍能被解读为有效输入的噪声量。如图1.23所示，低噪声容限和高噪声容限分别为：

$$NM_L = V_{IL} - V_{OL} \tag{1.2}$$

$$NM_H = V_{OH} - V_{IH} \tag{1.3}$$

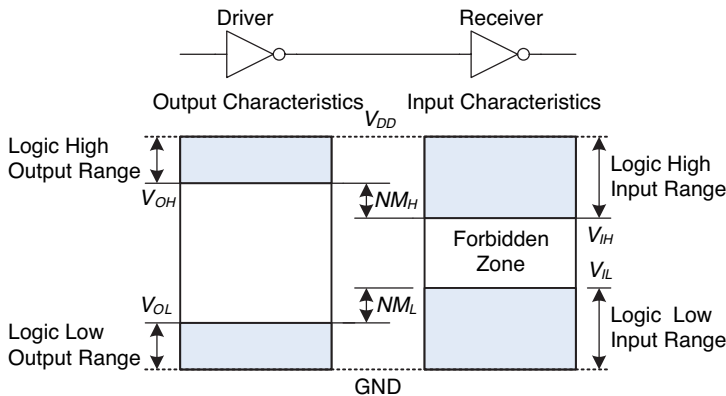


图1.23 逻辑电平与噪声容限

A	B	C	D	Y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

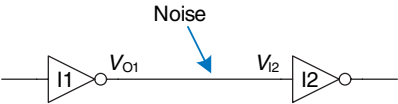
图1.22 四输入与门真值表

示例1.18 计算噪声容限

考虑图1.24中的反相器电路。 V_{O1} 是反相器I1的输出电压， V_{I2} 是反相器I2的输入电压。两个反相器具有以下特性： $V_{DD} = 5\text{ V}$ ， $V_{IL} = 1.35\text{ V}$ ， $V_{IH} = 3.15\text{ V}$ ， $V_{OL} = 0.33\text{ V}$ ， $V_{OH} = 3.84\text{ V}$ 。反相器的低噪声容限和高噪声容限分别是多少？该电路能否承受 V_{O1} 和 V_{I2} 之间1 V的噪声？

V_{DD} 代表{v*}上的电压
金属氧化物半导体晶体管的
drain，用于制造大多数现代
芯片。电源电压有时也称为
 V_{CC} ，代表旧技术中用于制造
芯片的双极型晶体管的
collector上的电压。地线有时
被称为 V_{SS} ，因为它是金属氧
化物半导体晶体管的source上
的电压。有关晶体管的更多信
息，请参见第1.7节。

图1.24 逆变器电路



解 反相器的噪声容限为： $NM_L = V_{IL} - V_{OL} = (1.35\text{ V} - 0.33\text{ V}) = 1.02\text{ V}$ ， $NM_H = V_{OH} - V_{IH} = (3.84\text{ V} - 3.15\text{ V}) = 0.69\text{ V}$ 。当输出为低电平时（ $NM_L = 1.02\text{ V}$ ），电路可承受1 V的噪声，但当输出为高电平时（ $NM_H = 0.69\text{ V}$ ）则不能。例如，假设驱动器I1输出其最差情况下的高电平值， $V_{OI} = V_{OH} = 3.84\text{ V}$ 。如果噪声导致电压在到达接收器输入端前下降1 V，则 $V_{I2} = (3.84\text{ V} - 1\text{ V}) = 2.84\text{ V}$ 。这低于可接受的输入高电平值， $V_{IH} = 3.15\text{ V}$ ，因此接收器可能无法检测到正确的高电平输入。

DC表示输入电压保持恒定或变化足够缓慢，使系统其余部分能够跟上的行为。该术语的历史根源来自*direct current*，一种通过恒定电压在线路上传输功率的方法。相比之下，电路的*transient*是输入电压快速变化时的行为。第2.9节将进一步探讨瞬态响应。

1.6.4 直流传输特性

要理解数字抽象的局限性，我们必须深入探究门电路的模拟行为。门电路的直流*transfer characteristics*描述了当输入变化足够慢以致输出能跟上时，输出电压作为输入电压的函数。这些被称为传输特性，因为它们描述了输入电压与输出电压之间的关系。

理想反相器在 $V_{DD}/2$ 处具有陡峭的开关阈值，如图1.25(a)所示。当 $V(A) < V_{DD}/2$ 时， $V(Y) = V_{DD}$ 。当 $V(A) > V_{DD}/2$ 时， $V(Y) = 0$ 。在这种情况下， $V_{IH} = V_{IL} = V_{DD}/2$ 。 $V_{OH} = V_{DD}$ 且 $V_{OL} = 0$ 。

实际逆变器在极端值之间的变化更为平缓，如图1.25(b)所示。当输入电压 $V(A)$ 为0时，输出电压 $V(Y) = V_{DD}$ 。当 $V(A) = V_{DD}$ 时， $V(Y) = 0$ 。然而，这些端点之间的过渡是平滑的，且可能并非精确居中于 $V_{DD}/2$ 。这就引出了如何定义逻辑电平的问题。

选择逻辑电平的一个合理位置是传输特性 $dV(Y)/dV(A)$ 斜率为-1的点。这两个点被称为*unity gain points*。在单位增益点选择逻辑电平通常能最大化噪声容限。如果 V_{IL} 减小， V_{OH} 只会小幅增加。但如果 V_{IL} 增大， V_{OH} 会急剧下降。

1.6.5 静态规则

为避免输入落入禁止区域，数字逻辑门被设计为遵循*static discipline*。静态约束要求，在给定逻辑有效输入的情况下，每个电路元件都将产生逻辑有效的输出。

通过遵循静态规范，数字设计者牺牲了使用任意模拟电路元件的自由，以换取

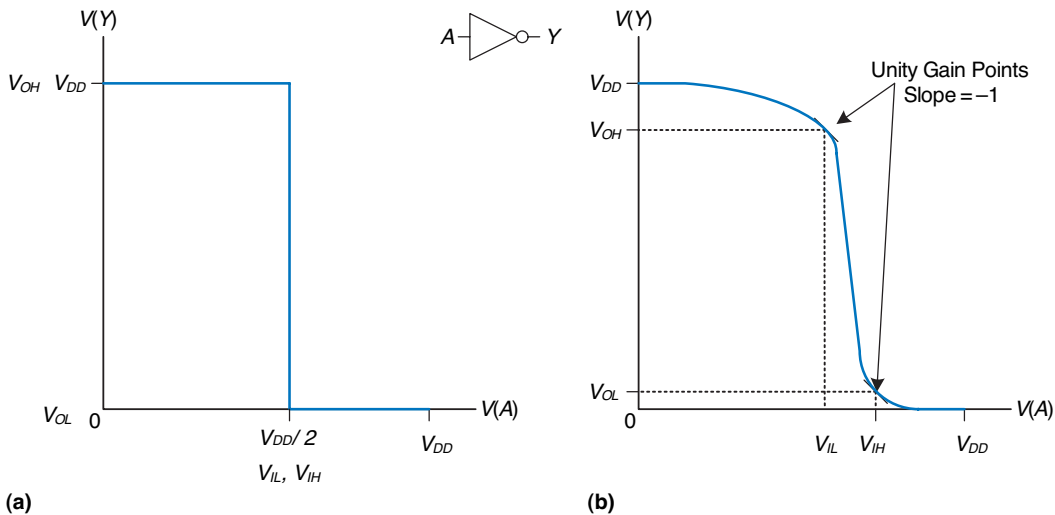


图1.25 直流传输特性与逻辑电平

数字电路的简洁性和鲁棒性。它们将抽象层次从模拟提升到数字，通过隐藏不必要的细节来提高设计效率。

V_{DD} 和逻辑电平的选择是任意的，但所有相互通信的门电路必须具有兼容的逻辑电平。因此，门电路被分组为*logic families*，使得同一逻辑系列中的所有门电路在与该系列其他门电路一起使用时遵循静态规范。同一逻辑系列中的逻辑门像乐高积木一样拼接在一起，因为它们使用一致的电源电压和逻辑电平。

从20世纪70年代到90年代占主导地位的四种主要逻辑系列是晶体管-晶体管逻辑（TTL）、互补金属氧化物半导体逻辑（CMOS，发音为sea-moss）、低电压TTL逻辑（LVTTTL）和低电压CMOS逻辑（LVCMOS）。它们的逻辑电平在表1.4中进行了比较。此后，逻辑系列随着更低电源电压的激增而分化。附录A.6更详细地重新讨论了流行的逻辑系列。

表1.4 5V和3.3V逻辑系列的电平

Logic Family	V_{DD}	V_{IL}	V_{IH}	V_{OL}	V_{OH}
TTL	5 (4.75–5.25)	0.8	2.0	0.4	2.4
CMOS	5 (4.5–6)	1.35	3.15	0.33	3.84
LVTTTL	3.3 (3–3.6)	0.8	2.0	0.4	2.4
LVCMOS	3.3 (3–3.6)	0.9	1.8	0.36	2.7

表1.5 逻辑系列的兼容性

Driver	Receiver			
	TTL	CMOS	LVTTL	LVC MOS
TTL	OK	NO: $V_{OH} < V_{IH}$	MAYBE ^a	MAYBE ^a
CMOS	OK	OK	MAYBE ^a	MAYBE ^a
LVTTL	OK	NO: $V_{OH} < V_{IH}$	OK	OK
LVC MOS	OK	NO: $V_{OH} < V_{IH}$	OK	OK

只要5V高电平不会损坏接收器输入

示例1.19 逻辑系列兼容性

表1.4中的哪些逻辑系列可以可靠地相互通信？

解决方案 表1.5列出了哪些逻辑系列具有兼容的逻辑电平。请注意，像TTL或CMOS这样的5V逻辑系列可能会产生高达5V的输出电压。如果这个5V信号驱动LVTTL或LVC MOS等3.3V逻辑系列的输入，除非接收器专门设计为“5伏兼容”，否则可能会损坏接收器。

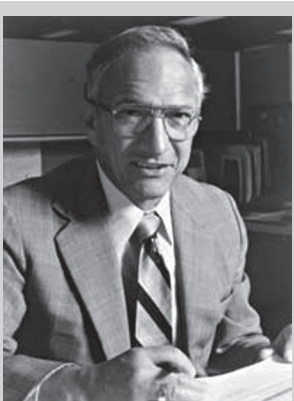
1.7 CMOS 晶体管*

本节及其他标有*的章节为可选内容，并非理解本书主线所必需。

巴贝奇的分析机由齿轮构成，早期电子计算机使用继电器或真空管。现代计算机采用晶体管，因其廉价、小巧且可靠。*Transistors* 是电控开关，当电压或电流施加到控制端时会导通或关断。两种主要晶体管类型为 *bipolar junction transistors* 和 *metal-oxide-semiconductor field effect transistors*（分别读作“莫斯菲”或“M-O-S”的 MOSFETs 或 MOS transistors）。

1958年，德州仪器的杰克·基尔比制造了首个包含两个晶体管的集成电路。1959年，仙童半导体的罗伯特·诺伊斯为在单个硅芯片上互连多个晶体管的方法申请了专利。当时，每个晶体管的价格约为10美元。

得益于三十多年来前所未有的制造进步，工程师如今可以在1平方厘米的硅芯片上集成约十亿个MOSFET，每个晶体管的成本不到10微美分。其容量和成本大约每8年提升一个数量级。MOSFET现已成为



罗伯特·诺伊斯，1927–1990年。出生于爱荷华州伯灵顿。在格林内尔学院获得物理学学士学位，并在麻省理工学院获得物理学博士学位。因其对行业的深远影响，被誉为“硅谷市长”。

1957年共同创立了飞兆半导体公司，1968年共同创立了英特尔公司。共同发明了集成电路。他团队中的许多工程师后来创立了其他开创性的半导体公司（照片©2006，英特尔公司。经许可转载）。

几乎所有数字系统。在本节中，我们将深入数字抽象层之下，了解如何用MOSFET构建逻辑门。

1.7.1 半导体

MOS晶体管由硅制成，硅是岩石和沙子中的主要原子。硅（Si）是第IV族原子，因此其价电子层有四个电子，并与四个相邻原子形成键，从而形成晶体 $lattice$ 。图1.26(a)为便于绘制以二维形式展示了晶格，但请记住晶格实际上形成立方晶体。图中每条线代表一个共价键。纯硅本身是良导体，因为所有电子都束缚在共价键中。然而，当小心添加少量称为 $dopant$ 原子的杂质时，其导电性会增强。如果加入第V族掺杂剂如砷（As），掺杂原子会多出一个不参与成键的电子。该电子可在晶格中自由移动，留下一个电离的掺杂原子（ As^+ ），如图1.26(b)所示。该电子携带负电荷，因此我们称砷为 $n-type$ 掺杂剂。另一方面，如果加入第III族掺杂剂如硼（B），掺杂原子会缺少一个电子，如图1.26(c)所示。这个缺失的电子称为 $hole$ 。邻近硅原子的一个电子可能移动过来填补缺失的键，形成电离的掺杂原子（ B^- ），并在邻近硅原子处留下一个空穴。类似地，空穴可在晶格中迁移。空穴是负电荷的缺失，因此其行为类似于带正电的粒子。因此，我们称硼为 $p-type$ 掺杂剂。由于硅的导电性随掺杂剂浓度变化可跨越多个数量级，硅被称为 $semiconductor$ 。

1.7.2 二极管

p型硅与n型硅之间的结称为 $diode$ 。p型区域称为 $anode$ ，n型区域称为 $cathode$ ，如图1.27所示。当阳极电压升高时

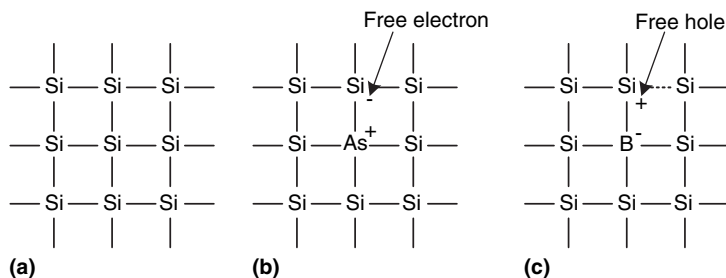


图1.26 硅晶格与掺杂原子

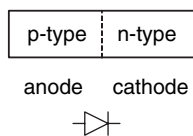


图1.27 p-n结二极管结构与符号



图1.28 电容器符号

当阳极电压高于阴极电压时，二极管处于 *forward biased* 状态，电流从阳极流向阴极。但当阳极电压低于阴极电压时，二极管处于 *reverse biased* 状态，没有电流通过。二极管的符号直观地表明电流仅单向流动。

1.7.3 电容器

capacitor 由两个被绝缘体隔开的导体组成。当电压 V 施加到其中一个导体上时，该导体积累电荷 Q ，另一个导体则积累相反的电荷 $-Q$ 。电容器的 *capacitance* C 是电荷与电压之比： $C = Q/V$ 。电容与导体的尺寸成正比，与它们之间的距离成反比。电容器的符号如图1.28所示。

电容之所以重要，是因为对导体充电或放电需要时间和能量。电容越大，电路运行速度越慢，所需能量也越多。本书将全程讨论速度与能量问题。

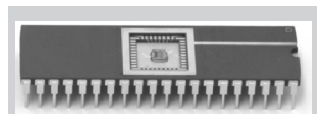
1.7.4 nMOS与pMOS晶体管

MOSFET是由多层导电和绝缘材料构成的夹层结构。MOSFET构建在直径约15至30厘米的薄而平坦的 *wafers* 硅片上。制造过程从裸晶圆开始，涉及一系列步骤：将掺杂剂注入硅中，生长二氧化硅和硅的薄膜，并沉积金属。每一步之间，晶圆会被 *patterned*，以确保材料仅出现在所需位置。由于晶体管的长度仅为亚微米级¹，且整个晶圆同时加工，因此一次制造数十亿个晶体管的成本很低。加工完成后，晶圆被切割成称为 *chips* 或 *dice* 的矩形，每个矩形包含数千、数百万甚至数十亿个晶体管。芯片经过测试后，被封装在带有金属引脚的塑料或陶瓷 *pack-age* 中，以便连接到电路板。

MOSFET 夹层结构由称为 *gate* 的导电层、位于其下的 *silicon dioxide* (SiO_2) 绝缘层以及底部的硅晶圆（称为 *substrate*）组成。历史上，栅极由金属制成，因此得名金属-氧化物-半导体。现代制造工艺使用多晶硅作为栅极材料，因其在后续高温处理步骤中不会熔化。二氧化硅更广为人知的名称为玻璃，通常简称为 *oxide*。



英特尔洁净室内的技术人员穿着 Gore-Tex 防尘服，以防止头发、皮肤和衣物上的微粒污染硅晶圆上的微型晶体管（照片© 2006，英特尔公司。经许可转载）。



一个40引脚双列直插式封装（DIP）在中心包含一个小芯片（几乎看不见），该芯片通过比头发丝还细的金线连接到40个金属引脚，每侧20个（照片由Kevin Mapp拍摄，© Harvey Mudd College）。

¹ $1\ \mu\text{m} = 1\ \text{micron} = 10^{-6}\ \text{m}$.

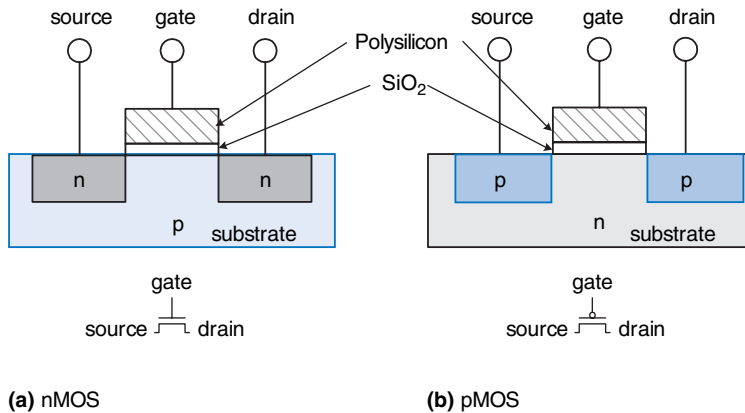


图1.29 nMOS和pMOS晶体管

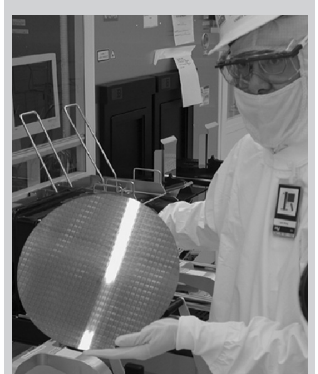
在半导体行业中，金属-氧化物-半导体三明治结构形成一个电容器，其中一层称为*dielectric*的薄绝缘氧化物将金属板和半导体板隔开。

MOSFET有两种类型：nMOS和pMOS（读作“n-莫斯”和“p-莫斯”）。图1.29展示了每种类型的横截面，是通过切割晶圆并从侧面观察得到的。n型晶体管称为nMOS，其栅极两侧有n型掺杂区域，分别称为*source*和*drain*，并构建在p型半导体衬底上。pMOS晶体管则正好相反，由n型*substrate*中的p型源极和漏极区域组成。

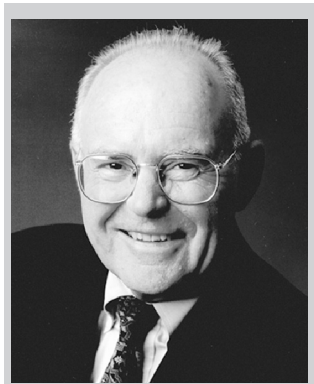
MOSFET 表现为一个电压控制开关，其中栅极电压产生电场，从而导通或关断源极与漏极之间的连接。术语 *field effect transistor* 即源于这一工作原理。让我们首先探讨 nMOS 晶体管的工作方式。

nMOS晶体管的衬底通常连接到系统最低电压GND。首先考虑栅极也处于0V的情况，如图1.30(a)所示。由于源极或漏极电压非负，源极与衬底之间以及漏极与衬底之间的二极管处于反向偏置状态。因此源极与漏极之间没有电流通路，晶体管处于关断状态。接下来考虑栅极电压升高至 V_{DD} 的情况，如图1.30(b)所示。当电容器上极板施加正电压时，会产生电场，吸引正电荷聚集在上极板，负电荷聚集在下极板。若电压足够大，栅极下方会吸引大量负电荷，使得区域*inverts*从p型有效转变为n型。这个反转区域称为*channel*。此时晶体管从n型源极经n型沟道到n型漏极形成连续通路，电子得以流动。

源极和漏极端子在物理上是对称的。然而，我们通常说电荷从源极流向漏极。在nMOS晶体管中，电荷由电子携带，电子从负电压流向正电压。在pMOS晶体管中，电荷由空穴携带，空穴从正电压流向负电压。如果我们将电路图绘制为最高电压在上方、最低电压在下方，那么nMOS晶体管中（负）电荷的源极是下方端子，而pMOS晶体管中（正）电荷的源极是上方端子。



一名技术人员手持一块12英寸的晶圆，上面包含数百个微处理器芯片（照片©2006，英特尔公司。经许可转载）。



戈登·摩尔，1929—出生于旧金山。在加州大学伯克利分校获得化学学士学位，并在加州理工学院获得化学与物理学博士学位。1968年与罗伯特·诺伊斯共同创立英特尔。1965年观察到计算机芯片上的晶体管数量每年翻一番。这一趋势被称为 *Moore's Law*。自1975年以来，晶体管数量每两年翻一番。

摩尔定律的一个推论是，微处理器性能每18到24个月翻一番。半导体销售额也呈指数级增长。不幸的是，功耗同样呈指数级增长。

摩尔定律推动半导体行业取得了50年的惊人进步，晶体管的特征尺寸从超过10 μm 缩小到仅5纳米。然而，由于制造远小于光波长的晶体管成本高昂，这一进展已放缓。（照片©2006，英特尔公司。经许可转载。）

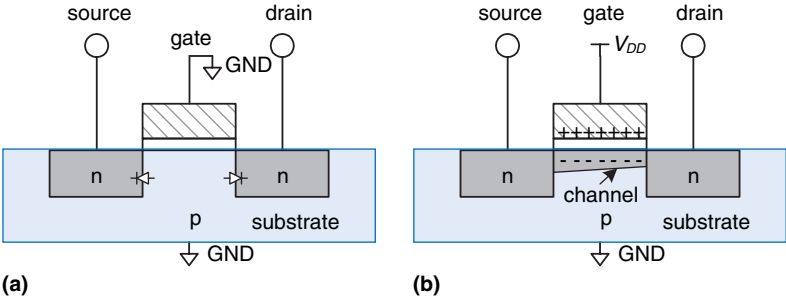


图1.30 nMOS晶体管工作原理

源极到漏极。晶体管导通。使晶体管导通所需的栅极电压称为 *threshold voltage*、 V_t ，通常为0.3至0.7 V。

pMOS 晶体管的工作方式正好相反，从图 1.31 中其符号上的气泡可以推测出来。衬底连接到 V_{DD} 。当栅极也处于 V_{DD} 时，pMOS 晶体管关断。当栅极处于 GND 时，沟道反转为 p 型，pMOS 晶体管导通。

遗憾的是，MOSFET并非理想的开关。具体而言，nMOS晶体管能良好地传输0，但传输1的效果较差。当nMOS晶体管的栅极电压为 V_{DD} 时，若其漏极电压在0到 V_{DD} 之间变化，源极电压仅能在0到 $V_{DD} - V_t$ 之间摆动。类似地，pMOS晶体管能良好地传输1，但传输0的效果较差。不过，我们将看到，可以构建仅利用晶体管良好工作模式的逻辑门。

nMOS晶体管需要p型衬底，而pMOS晶体管需要n型衬底。为了在同一芯片上制造两种类型的晶体管，制造工艺通常从p型晶圆开始，然后在需要放置pMOS晶体管的位置注入称为 *wells* 的n型区域。这些提供两种类型晶体管的工艺被称为互补MOS或CMOS。CMOS工艺用于制造当今绝大多数晶体管。

总之，CMOS工艺为我们提供了两种电控开关，如图1.31所示。电压在

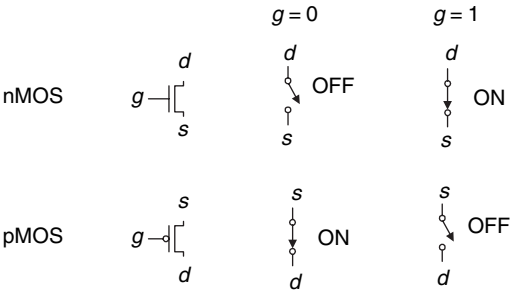


图1.31 MOSFET的开关模型

栅极 (g) 控制源极 (s) 和漏极 (d) 之间的电流流动。nMOS晶体管在栅极为0时关闭，在栅极为1时开启。pMOS晶体管则相反：栅极为0时开启，栅极为1时关闭。

1.7.5 CMOS非门

图1.32展示了一个由CMOS晶体管构建的非门原理图。三角形表示GND，横条表示 V_{DD} ；在后续的原理图中将省略这些标注。nMOS晶体管N1连接在GND与Y输出端之间。pMOS晶体管P1连接在 V_{DD} 与Y输出端之间。两个晶体管的栅极均由输入A控制。

如果 $A = 0$ ，则 N1 关断，P1 导通。因此，Y 连接到 V_{DD} 但未接地，并被上拉至逻辑 1。P1 传递良好的 1。如果 $A = 1$ ，则 N1 导通，P1 关断，Y 被下拉至逻辑 0。N1 传递良好的 0。对照图 1.12 中的真值表检查，可以看出该电路确实是一个非门。

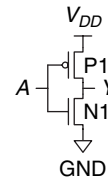


图1.32 非门原理图

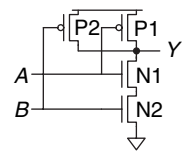


图1.33 双输入与非门原理图

1.7.6 其他CMOS逻辑门

图1.33展示了一个两输入与非门的示意图。在电路图中，导线总是在三叉连接处相连。只有在显示圆点的情况下，导线才会在四叉连接处相连。nMOS晶体管N1和N2串联；两个nMOS晶体管都必须导通才能将输出拉低至GND。pMOS晶体管P1和P2并联；只需一个pMOS晶体管导通即可将输出拉高至 V_{DD} 。表1.6列出了下拉网络和上拉网络的工作状态以及输出的状态，表明该门电路确实具有与非功能。例如，当 $A = 1$ 且 $B = 0$ 时，N1导通，但N2关断，阻断了从Y到GND的路径。P1关断，但P2导通，形成了从 V_{DD} 到Y的路径。因此，Y被上拉至1。

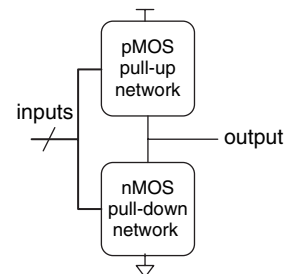


图1.34 反相逻辑门的一般形式

图1.34展示了用于构建任何反相逻辑门（如非门、与非门或非门）的通用形式。nMOS晶体管擅长传递0，因此将nMOS晶体管的下拉网络置于

表1.6 与非门操作

A	B	Pull-Down Network	Pull-Up Network	Y
0	0	OFF	ON	1
0	1	OFF	ON	1
1	0	OFF	ON	1
1	1	ON	OFF	0

经验丰富的设计师声称，电子设备之所以能运行，是因为它们含有*magic smoke*。他们通过观察证实了这一理论：一旦设备中的魔法烟雾泄漏出来，它就会停止工作。

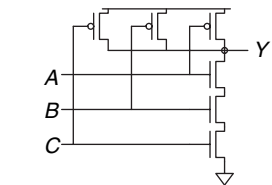


图1.35 三输入与非门原理图

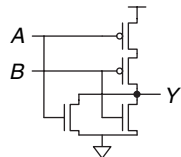


图1.36 双输入或非门原理图

在输出端与GND之间放置nMOS晶体管下拉网络，将输出拉低至0。pMOS晶体管擅长传递1，因此在输出端与 V_{DD} 之间放置pMOS晶体管上拉网络，将输出拉高至1。这些网络可由串联或并联的晶体管组成。当晶体管并联时，只要任一晶体管导通，网络即导通；当晶体管串联时，仅当两个晶体管均导通时网络才导通。输入线上的斜线表示栅极可能接收多个输入信号。

如果上拉网络和下拉网络同时导通，则 V_{DD} 与GND之间会存在一个 *short circuit*。门的输出可能处于禁止区，并且晶体管会消耗大量功率，甚至可能足以烧毁。另一方面，如果上拉网络和下拉网络同时关断，则输出既不连接到 V_{DD} 也不连接到GND。我们说输出*floats*。其值同样未定义。浮空输出通常是不希望的，但在第2.6节中，我们将看到它们有时如何能被设计者利用。

在正常工作的逻辑门中，任一时刻应有一个网络导通，另一个网络关断，从而使输出被拉高或拉低，既不会短路也不会悬空。我们可以通过*conduction complements*规则来保证这一点：当nMOS晶体管串联时，pMOS晶体管必须并联；当nMOS晶体管并联时，pMOS晶体管必须串联。

示例1.20 三输入与非门原理图

使用CMOS晶体管绘制三输入与非门的原理图。

解决方案 NAND门应仅在三个输入均为1时输出0。因此，下拉网络应由三个串联的nMOS晶体管组成。根据导通互补规则，pMOS晶体管必须并联。此类门电路如图1.35所示；您可以通过验证其真值表是否正确来确认其功能。

示例1.21 双输入或非门原理图

使用CMOS晶体管绘制一个两输入或非门的原理图。

解决方案 NOR门应在任一输入为1时输出0。因此，下拉网络应包含两个并联的nMOS晶体管。根据导通互补规则，pMOS晶体管必须串联。此类门电路如图1.36所示。

示例1.22 双输入与原理图

绘制一个双输入与门的原理图。

解决方案 使用单个CMOS门构建与门是不可能的。然而，构建与非门和非门很容易。因此，使用CMOS晶体管构建与门的最佳方法是采用与非门后接非门的方式，如图1.37所示。



图1.37 双输入与门原理图

1.7.7 传输门

有时，设计者会发现使用一种既能良好传递0又能良好传递1的理想开关非常方便。回想一下，nMOS晶体管擅长传递0，而pMOS晶体管擅长传递1，因此两者的并联组合能够很好地传递这两种值。图1.38展示了这样一个电路，称为*transmission gate*或*pass gate*。开关的两侧被称为A和B，因为开关是双向的，没有固定的输入或输出侧。控制信号被称为*enables*、EN和 $\overline{EN}$ 。当EN = 0且 $\overline{EN}$ = 1时，两个晶体管都处于关闭状态。因此，传输门关闭或禁用，A和B不连接。当EN = 1且 $\overline{EN}$ = 0时，传输门开启或启用，任何逻辑值都可以在A和B之间流动。

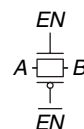


图1.38 传输门

1.7.8 伪nMOS逻辑

一个N输入的CMOS或非门使用N个并联的nMOS晶体管和N个串联的pMOS晶体管。串联的晶体管比并联的晶体管慢，就像串联的电阻比并联的电阻具有更大的电阻值一样。此外，pMOS晶体管比nMOS晶体管慢，因为空穴在硅晶格中的移动速度不如电子快。因此，并联的nMOS晶体管速度快，而串联的pMOS晶体管速度慢，尤其是当多个晶体管串联时。

伪nMOS逻辑用单个始终导通的弱pMOS晶体管替代了缓慢的pMOS晶体管堆栈，如图1.39所示。该pMOS晶体管通常被称为*weak pull-up*。pMOS晶体管的物理尺寸经过选择，使其仅在没有任何nMOS晶体管导通时，才能将输出Y弱上拉至高电平。但若有任何nMOS晶体管导通，它将压倒弱上拉，并将Y下拉至接近GND以产生逻辑0。

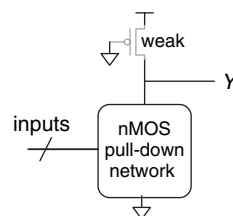


图1.39 通用伪nMOS门

伪nMOS逻辑的优势在于可用于构建具有多输入端的快速NOR门。例如，图1.40展示了一个伪nMOS四输入NOR门。伪nMOS门对于第5章讨论的某些存储器和逻辑阵列非常有用。其缺点是

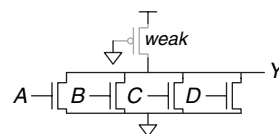


图1.40 伪nMOS四输入或非门

当输出为低电平时, V_{DD} 与GND之间存在短路; 弱pMOS和nMOS晶体管均导通。该短路会持续消耗功率, 因此伪nMOS逻辑必须谨慎使用。

伪nMOS门得名于20世纪70年代, 当时制造工艺仅有nMOS晶体管。由于没有pMOS晶体管, 便使用一个弱nMOS晶体管将输出上拉至高电平。

1.8 功耗*

*Power consumption*是单位时间内使用的能量。功耗在数字系统中至关重要。手机和笔记本电脑等便携式系统的电池寿命受功耗限制。例如, 手机电池大约储存10瓦时(W-hr)的能量, 这意味着它可以提供1瓦(W)的功率持续10小时, 或2瓦持续5小时, 以此类推。要使手机电池持续一整天, 其平均功耗应低于1瓦。笔记本电脑通常配备50至100瓦时的电池, 正常运行时功耗低于10瓦, 其中屏幕占很大一部分。对于插电使用的系统, 功耗也很重要, 因为电力需要花钱并产生排放, 而且如果系统消耗过多功率, 会过热。一台台式电脑每天运行8小时, 功耗为200瓦, 每年大约消耗600千瓦时(kW-hr)的电力。按每千瓦时12美分的平均成本和1磅CO₂排放计算, 这相当于每年72美元的电费以及600磅CO₂排放。

确保电脑在不使用时进入休眠状态, 将有助于保护地球。

数字系统同时消耗*dynamic*和*static*功率。动态功率是信号在0和1之间变化时用于对电容充电的功率。静态功率是即使信号不变且系统空闲时也会消耗的功率。

逻辑门及其连接导线具有电容。将电容C充电至电压 V_{DD} 时, 从电源汲取的能量为 CV_{DD}^2 。若系统以*frequency* f 频率运行, 且电容充放电周期占比为 α (称为*activity factor*), 则动态功耗为

$$P_{\text{dynamic}} = \alpha CV_{DD}^2 f \quad (1.4)$$

图1.41展示了活动因子。图1.41(a)显示了一个时钟信号, 每个周期上升和下降一次, 因此其活动因子为1。从一个上升沿到下一个上升沿的*clock period*称为 T_c (即*cycle time*), 它是时钟频率 f 的倒数。图1.41(b)显示了一个每个时钟周期切换一次的数据信号。时序图上的平行线表示该信号可能为高电平或低电平; 我们并不关心其具体值。交叉点表示该信号在每个时钟周期早期变化一次。因此, 该活动

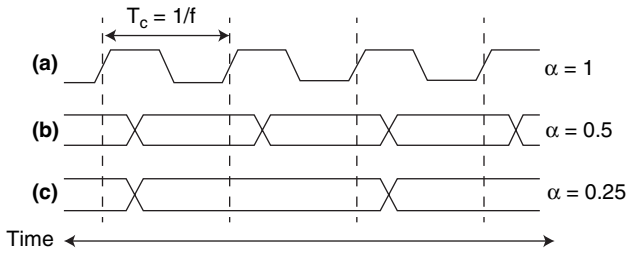


图1.41 活动因子示意图

因子为0.5（一半周期上升，一半周期下降）。图1.41(c)显示了一个随机数据信号，该信号在一半周期内切换，在另一半周期内保持不变。因此，其活动因子为0.25。实际的数字系统通常包含不进行切换的空闲组件，因此0.1的活动因子更为典型。

电气系统在空闲时也会消耗一定电流。当晶体管关闭时，会泄漏少量电流。某些电路（如第1.7.8节讨论的伪nMOS门）存在一条从 V_{DD} 到GND的路径，电流持续流过该路径。总静态电流 I_{DD} 也称为*leakage current*或*quiescent supply current*，在 V_{DD} 与GND之间流动。静态功耗与该静态电流成正比：

$$P_{\text{static}} = I_{DD}V_{DD} \quad (1.5)$$

示例1.23 功耗

某款手机配备8瓦时电池，工作电压为0.707伏。假设使用时手机运行频率为2 GHz。电路总电容为10纳法（ 10^{-8} 法拉），活动因子为0.05。当语音或数据处于活动状态（占使用时间的10%）时，天线还会发射3瓦功率。手机不使用时，由于信号处理关闭，动态功耗几乎降至零。但无论是否使用，手机都会消耗100毫安的静态电流。请计算手机在以下情况下的电池续航时间：（a）未使用状态；（b）持续使用状态。

解决方案 静态功率为 $P_{\text{static}} = (0.100 \text{ A})(0.707 \text{ V}) = 71 \text{ 毫瓦 (mW)}$ 。（a）如果手机未被使用，这是唯一的功耗，因此电池寿命为 $(8 \text{ W-hr}) / (0.071 \text{ W}) = 113 \text{ 小时}$ （约5天）。（b）如果手机正在使用，动态功率为 $P_{\text{dynamic}} = (0.05)(10^{-8} \text{ F})(0.707 \text{ V})^2(2 \times 10^9 \text{ Hz}) = 0.5 \text{ W}$ 。平均广播功率为 $(3 \text{ W})(0.1) = 0.3 \text{ W}$ 。

结合静态功率和广播功率，总有功率为 $0.5 \text{ W} + 0.071 \text{ W} + 0.3 \text{ W} = 0.871 \text{ W}$ ，因此电池续航时间为 $8 \text{ W-hr} / 0.871 \text{ W} \approx 9.2 \text{ 小时}$ 。这个例子在一定程度上简化了手机的实际运行情况，但它说明了功耗的关键概念。

1.9 总结与展望

There are 10 kinds of people in this world: those who can count in binary and those who can't.

本章介绍了理解 and 设计复杂系统的原理。尽管现实世界是模拟的，但数字设计者约束自己使用可能信号的一个离散子集。特别是，二进制变量只有两种状态：0和1，也称为FALSE和TRUE或LOW和HIGH。逻辑门根据一个或多个二进制输入计算二进制输出。一些常见的逻辑门包括：

非 (NOT)：当输入为假 (FALSE) 时输出为真 (TRUE) ▶ $\neg A$
与 (AND)：当所有输入为真 (TRUE) 时输出为真 (TRUE) ▶ $A \wedge B$
或 (OR)：当任一输入为真 (TRUE) 时输出为真 (TRUE) ▶ $A \vee B$
异或 (XOR)：当奇数个输入为真 (TRUE) 时输出为真 (TRUE)

逻辑门通常由CMOS晶体管构成，这些晶体管表现为电控开关。nMOS晶体管在栅极为1时导通，pMOS晶体管在栅极为0时导通。

第2章至第5章继续研究数字逻辑。第2章讨论 *combinational logic*，其输出仅取决于当前输入。已介绍的逻辑门就是组合逻辑的示例。你将学习设计包含多个门的电路，以实现由真值表或布尔方程指定的输入输出关系。第3章讨论 *sequential logic*，其输出同时取决于当前和过去的输入。*Registers* 是常见的时序元件，能够记忆先前的输入。*Finite state machines* 由寄存器和组合逻辑构建，是一种以系统化方式构建复杂系统的强大方法。我们还将研究数字系统的时序，以分析系统能运行多快。第4章介绍硬件描述语言 (HDL)。HDL 与传统编程语言相关，但用于模拟和构建硬件而非软件。如今大多数数字系统都使用HDL设计。SystemVerilog和VHDL是两种主流语言，本书将并行介绍它们。第5章研究其他组合和时序构建模块，如加法器、乘法器和存储器。

第六章转向计算机体系结构。它描述了RISC-V处理器，这是一种近期开发的开源微处理器，开始在工业界和学术界得到广泛应用。RISC-V架构由其寄存器和汇编语言指令集定义。你将

学习使用RISC-V处理器的汇编语言编写程序，以便能够以其原生语言与处理器进行通信。

第7章和第8章弥合了数字逻辑与计算机体系结构之间的鸿沟。第7章探讨了微架构，即构建处理器所需的数字积木（如加法器和寄存器）的布局。在这一章中，你将学习构建自己的RISC-V处理器。实际上，你将学习三种微架构，它们展示了性能与成本之间的不同权衡。处理器性能呈指数级增长，需要日益复杂的存储系统来满足对数据永不满足的需求。第8章深入探讨了存储系统架构。第9章（作为网络补充材料提供——见前言）描述了计算机如何与外围设备（如显示器、蓝牙无线电和电机）进行通信。

练习

练习1.1 请用一段话解释至少三个由

- (a) 研究细胞运作的生物学家。
- (b) 研究物质组成的化学家。

练习1.2 用一段话解释层次化、模块化和规律性技术如何被用于

- (a) 汽车设计师。
- (b) 企业管理其运营。

练习1.3 Ben Bitdiddle正在建造一栋房子。解释他如何利用层次化、模块化和规范化的原则在施工过程中节省时间和金钱。

习题1.4 一个模拟电压的范围为0–5 V。若能以 ± 50 mV的精度进行测量，它最多能传递多少比特的信息？

练习1.5 教室墙上有一个旧钟，它的分针断了。

- (a) 如果你能读取时针并精确到15分钟，那么时钟传递了多少比特的时间信息？
- (b) 如果你知道是上午还是下午，那么关于时间你额外知道了多少比特的信息？

练习1.6 巴比伦人大约在4000年前发展了*sexagesimal*（六十进制）数字系统。一个六十进制数字传递了多少比特的信息？如何用六十进制表示数字 4000_{10} ？

练习1.7 一个十进制数用1可以表示多少个不同的数字？6位？

练习1.8 最大的无符号32位二进制数是多少？

练习1.9 可以用16位二进制表示的最大数是多少？

- (a) 无符号数？
- (b) 二进制补码数？
- (c) 符号/数值表示法？

练习1.10 可以用32位二进制表示的最大数是多少

- (a) 无符号数? (b) 二进制补码数
? (c) 符号/数值数?

练习1.11 可以用最小的（最负的）16位二进制数表示什么

- (a) 无符号数? (b) 二进制补码数
? (c) 符号/数值数?

练习1.12 可以用32位二进制表示的最小（最负）数是多少

- (a) 无符号数? (b) 二进制补码数
? (c) 符号/数值数?

练习1.13 将以下无符号二进制数转换为十进制。请展示你的计算过程。

- (a) 1010_2 (b) 110110_2 (c)
 11110000_2 (d) 000100010
 100111_2

练习1.14 将以下无符号二进制数转换为十进制。请展示你的计算过程。

- (a) 1110_2 (b) 100100_2 (c)
 11010111_2 (d) 011101010
 100100_2

练习1.15 重复练习1.13, 但转换为十六进制

1.

练习1.16 重复练习1.14, 但转换为十六进制。

练习1.17 将下列十六进制数转换为十进制。展示你的计算过程。

(a) $A5_{16}$ (b) $3B_{16}$
(c) $FFFF_{16}$ (d) $D0$
 000000_{16}

练习1.18 将下列十六进制数转换为十进制。展示你的计算过程。

(a) $4E_{16}$ (b) $7C_{16}$
(c) $ED3A_{16}$ (d) 40
 $3FB001_{16}$

练习1.19 重复练习1.17，但转换为无符号二进制。

练习1.20 重复练习1.18，但转换为无符号二进制。

练习1.21 将下列二进制补码数转换为十进制。

(a) 1010_2 (b) 110
 110_2 (c) 011100
 00_2 (d) 1001111
 1_2

练习1.22 将下列二进制补码数转换为十进制。

(a) 1110_2 (b) 100
 011_2 (c) 010011
 10_2 (d) 1011010
 1_2

练习1.23 重复练习1.21，假设二进制数为符号/数值形式而非二进制补码表示。
。

练习1.24 重复练习1.22，假设二进制数为符号/数值形式而非二进制补码表示。
。

练习1.25 将下列十进制数转换为无符号二进制数。

(a) 42_{10} (b)
) 63_{10} (c) 2
 29_{10} (d) 84
 5_{10}

练习1.26 将下列十进制数转换为无符号二进制数。

(a) 14_{10} (b)
) 52_{10} (c) 3
 39_{10} (d) 71
 1_{10}

练习1.27 重复练习1.25，但转换为十六进制。

练习1.28 重复练习1.26，但转换为十六进制。

练习1.29 将下列十进制数转换为8位二进制补码数，或指出该十进制数会超出表示范围。

(a) 42_{10} (b)
 -63_{10} (c) 12
 4_{10} (d) -128
 10 (e) 133_{10}

练习1.30 将下列十进制数转换为8位二进制补码数，或指出该十进制数会溢出范围。

(a) 24_{10} (b)
 -59_{10} (c) 12
 8_{10} (d) -150
 10 (e) 127_{10}

练习1.31 重复练习1.29，但转换为8位符号/数值表示的数字。

练习1.32 重复练习1.30, 但转换为8位符号/数值表示法。

练习1.33 将以下4位二进制补码数转换为8位二进制补码数。

(a) 0101_2 (

b) 1010_2

练习1.34 将以下4位二进制补码数转换为8位二进制补码数。

(a) 0111_2 (

b) 1001_2

练习1.35 如果数字是无符号数而非二进制补码, 重复练习1.33。

练习1.36 如果数字是无符号数而非二进制补码, 重复练习1.34。

练习1.37 基数为8的记数法称为*octal*。将练习1.25中的每个数字转换为八进制。

练习1.38 八进制被称为*octal*。将练习1.26中的每个数字转换为八进制。

练习1.39 将下列八进制数转换为二进制、十六进制和十进制。

(a) 42_8 (b)

63_8 (c) 255

$_8$ (d) 3047_8

练习1.40 将下列每个八进制数转换为二进制、十六进制和十进制。

(a) 23_8 (b)

45_8 (c) 371

$_8$ (d) 2560_8

练习1.41 有多少个5位二进制补码数大于0? 有多少个小于0? 对于符号/数值表示法, 你的答案会有什么不同?

练习1.42 有多少个7位二进制补码数大于0？有多少个小于0？对于符号/数值表示法，你的答案会有什么不同？

练习1.43 一个32位字中有多少字节？一个32位字中有多少半字节？

练习1.44 一个64位字包含多少字节？

练习1.45 某特定DSL调制解调器以768 kbits/sec的速率运行。它在1分钟内能接收多少字节？

练习1.46 USB 3.0每秒可发送5 Gbits数据。请问1分钟内它能发送多少字节？

练习1.47 硬盘制造商使用术语“兆字节”表示 10^6 字节，“千兆字节”表示 10^9 字节。在一个50 GB的硬盘上，你可以存储多少真正的GB（即GiB）的音乐？

练习1.48 不使用计算器估算 2^{31} 的值。

练习1.49 奔腾II微处理器中的存储器被组织成一个矩形位阵列，具有 2^8 行和 2^9 列。在不使用计算器的情况下估算其位数。

练习1.50 绘制一个类似于图1.11的数轴，用于3位无符号数、二进制补码数和符号/数值数。

练习1.51 绘制一个类似于图1.11的数轴，用于表示2位无符号数、二进制补码数和符号/数值数。

练习1.52 执行以下无符号二进制数的加法。指出和是否溢出4位结果。

(a) $1001_2 + 0100_2$ (b)
) $1101_2 + 1011_2$

练习1.53 执行以下无符号二进制数的加法。指出和是否溢出8位结果。

(a) $10011001_2 + 01000100_2$ (b)
 $11010010_2 + 10110110_2$

练习1.54 重复练习1.52，假设二进制数为补码形式。

练习1.55 重复练习1.53，假设二进制数为补码形式。

练习1.56 将下列十进制数转换为6位二进制补码并相加。指出结果是否超出6位范围产生溢出。

- (a) $16_{10} + 9_{10}$ (b) $27_{10} + 31_{10}$ (c) $-4_{10} + 19_{10}$ (d) $3_{10} + -32_{10}$ (e) $-16_{10} + -9_{10}$ (f) $-27_{10} + -31_{10}$

练习1.57 对以下数字重复练习1.56。

- (a) $7_{10} + 13_{10}$ (b) $17_{10} + 25_{10}$ (c) $-26_{10} + 8_{10}$ (d) $31_{10} + -14_{10}$ (e) $-19_{10} + -22_{10}$ (f) $-2_{10} + -29_{10}$

练习1.58 执行以下无符号十六进制数的加法。指出和是否溢出8位（两位十六进制数字）结果。

- (a) $7_{16} + 9_{16}$ (b) $13_{16} + 28_{16}$ (c) $AB_{16} + 3E_{16}$ (d) $8F_{16} + AD_{16}$

练习1.59 执行以下无符号十六进制数的加法。指出和是否溢出8位（两位十六进制数字）结果。

- (a) $22_{16} + 8_{16}$ (b) $73_{16} + 2C_{16}$ (c) $7F_{16} + 7F_{16}$ (d) $C2_{16} + A4_{16}$

练习1.60 将下列十进制数转换为5位二进制补码形式并进行减法运算。指出差值是否溢出5位结果。

- (a) $9_{10} - 7_{10}$ (b) $12_{10} - 15_{10}$

- (c) $-6_{10} - 11_{10}$ (d) $4_{10} - -8_{10}$

练习1.61 将下列十进制数转换为6位二进制补码并相减。指出差值是否溢出6位结果。

- (a) $18_{10} - 12_{10}$ (b) $30_{10} - 9_{10}$ (c) -2
 $8_{10} - 3_{10}$ (d) -16
 $10 - 21_{10}$

练习1.62 在一个偏移量为 B 的 $biased\ N$ 位二进制数系统中，正数和负数表示为它们的值加上偏移量 B 。例如，对于偏移量为15的5位数，数字0表示为01111，1表示为10000，依此类推。偏移数系统有时用于浮点数学中，这将在第5章讨论。考虑一个偏移量为 127_{10} 的8位二进制数系统。

- (a) 二进制数 10000010_2 表示的十进制值是多少？ (b) 表示数值 0 的二进制数是什么？ (c) 最负数的表示形式及其值是什么？ (d) 最正数的表示形式及其值是什么？

练习1.63 绘制一个类似于图1.11的数轴，用于3位偏移量为3的偏移码（偏移码的定义见练习1.62）。

练习1.64 在 $binary\ coded\ decimal$ (BCD) 系统中，使用4位二进制数来表示0到9的十进制数字。例如， 37_{10} 被写作 00110111_{BCD} 。

- (a) 将 289_{10} 转换为BCD码。 (b) 将 100101010001_{BCD} 转换为十进制。
 (c) 将 01101001_{BCD} 转换为二进制。 (d) 解释为什么BCD可能是表示数字的一种有用方式。

练习1.65 回答以下与BCD系统相关的问题（BCD的定义见练习1.64）。

- (a) 将 371_{10} 写成BCD码。 (b) 将 000110000111_{BCD} 转换为十进制。 (c) 将 10010101_{BCD} 转换为二进制。 (d) 解释BCD码与二进制表示数字相比的缺点。

习题1.66 一艘飞碟在内布拉斯加州的玉米地坠毁。联邦调查局调查残骸时发现一本工程手册，其中包含一个火星数字系统中的方程： $325 + 42 = 411$ 。如果这个方程成立，你预计火星人会几根手指？

练习1.67 Ben Bitdiddle和Alyssa P. Hacker正在争论。Ben说：“所有大于零且能被六整除的整数，其二进制表示中恰好有两个1。” Alyssa不同意。她说：“不对，但所有这些数的二进制表示中1的个数都是偶数。”你同意Ben还是Alyssa，还是两者都同意，或者都不同意？请解释。

练习1.68 Ben Bitdiddle和Alyssa P. Hacker又吵起来了。Ben说：“我可以通过先减1，然后对结果的所有位取反来得到一个数的二进制补码。” Alyssa说：“不对，我可以从最低有效位开始检查这个数的每一位。当找到第一个1时，对后续的每一位取反。”你同意Ben还是Alyssa，还是两者都同意，或者都不同意？请解释。

练习1.69 用你喜欢的语言（例如C、Java、Perl）编写一个程序，将数字从二进制转换为十进制。用户应输入一个无符号二进制数。程序应输出对应的十进制数。

练习1.70 重复练习1.69，但将用户指定的任意基数 b_1 转换为另一基数 b_2 。支持最高16进制，使用字母表示大于9的数字。用户应输入 b_1 、 b_2 ，然后输入要转换的基数 b_1 下的数字。程序应输出基数 b_2 下的等效数字。

练习1.71 画出符号、布尔方程和真值表

- (a) 一个三输入或门 (b) 一个三输入异或门 (c) 一个四输入同或门

练习1.72 画出符号、布尔方程和真值表

- (a) 一个四输入或门 (b) 一个三输入异或非门 (c) 一个五输入与非门

练习1.73 一个 *majority gate* 当且仅当其超过一半的输入为TRUE时输出TRUE。完成图1.42所示的三输入多数门的真值表。

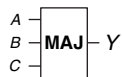


图1.42 三输入多数门

练习1.74 图1.43所示的三输入AND-OR (AO) *gate*, 如果A和B均为真或C为真, 则输出为真。请完成该门的真值表。



图1.43 三输入与或门

练习1.75 一个三输入OR-AND-INVERT (OAI) *gate*如图1.44所示, 如果C为真且A或B为真, 则输出FALSE。否则, 输出TRUE。请完成该门的真值表。

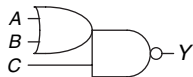


图1.44 三输入或与反相门

练习1.76 对于两个变量的布尔函数, 存在16种不同的真值表。列出每种真值表, 并为每个真值表取一个简短的描述性名称 (例如OR、NAND等)。

练习1.77 对于N个变量的布尔函数, 存在多少种不同的真值表?

练习1.78 是否可能分配逻辑电平, 使得具有图1.45所示传输特性的器件可用作反相器? 如果可以, 输入和输出的低电平与高电平 (V_{IL} 、 V_{OL} 、 V_{IH} 和 V_{OH}) 以及噪声容限 (NM_L 和 N_H) 分别是多少? 如果不行, 请解释原因。

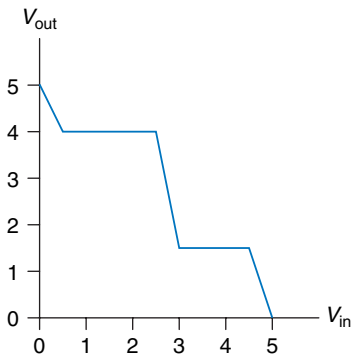


图1.45 直流传输特性

练习1.79 对图1.46所示的传输特性重复练习1.78。

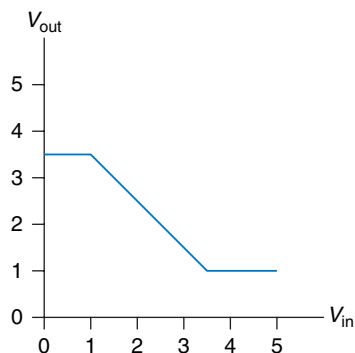


图1.46 直流传输特性

练习1.80 是否可能分配逻辑电平，使得具有图1.47所示传输特性的器件能够用作缓冲器？如果可以，输入和输出的低电平与高电平（ V_{IL} 、 V_{OL} 、 V_{IH} 和 V_{OH} ）以及噪声容限（ NM_L 和 NM_H ）分别是多少？如果不行，请解释原因。

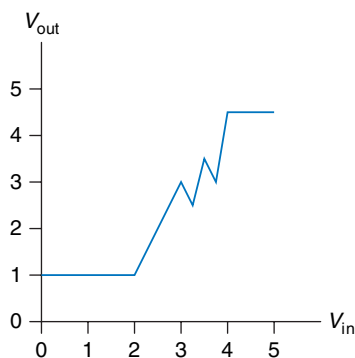


图1.47 直流传输特性

练习1.81 Ben Bitdiddle发明了一个电路，其传输特性如图1.48所示，他希望将其用作缓冲器。它能正常工作吗？为什么能或为什么不能？他希望宣称该电路与LVCMOS和LVTTTL逻辑兼容。Ben的缓冲器能否正确接收来自这些逻辑系列的输入？其输出能否正确驱动这些逻辑系列？请解释。

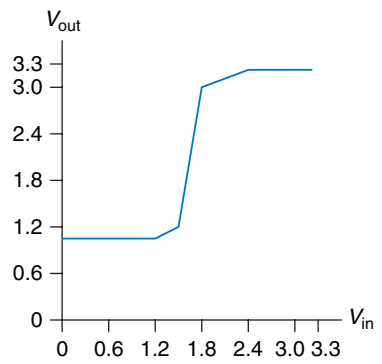


图1.48 Ben缓冲器的直流传输特性

练习1.82 本·比特迪德尔在一条黑暗的小巷中行走时，遇到了一个具有图1.49所示传输函数的两输入门。输入为A和B，输出为Y。

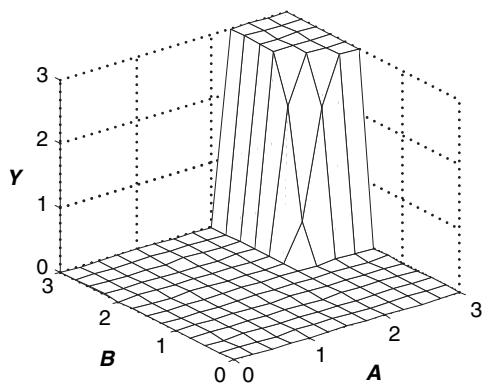


图1.49 双输入直流传输特性

- (a) 他找到了哪种逻辑门？
- (b) 近似的高逻辑电平和低逻辑电平是多少？

练习1.83 对图1.50重复练习1.82。

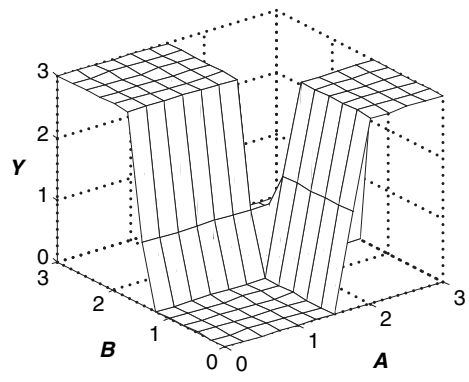


图1.50 双输入直流传输特性

练习1.84 画出以下CMOS门的晶体管级电路。
使用最少数量的晶体管。

(a) 四输入与非门 (b) 三输入或与反相门（见练习1.75） (c)
三输入与或门（见练习1.74）

练习1.85 画出以下CMOS门电路的晶体管级电路图。使用最少数量的晶体管。

(a) 三输入或非门 (b) 三输入与门 (c) 二输入或门

练习1.86 一个 *minority gate* 在且仅当其输入中少于一半为TRUE时输出TRUE，否则输出FALSE。请设计一个三输入CMOS少数门的晶体管级电路，使用最少晶体管数量。

练习1.87 写出图1.51中门电路所执行功能的真值表。该真值表应有两个输入A和B。这个函数的名称是什么？

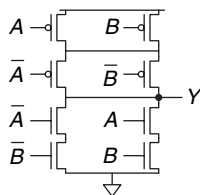


图1.51 神秘原理图

练习1.88 写出图1.52中门电路所执行功能的真值表。该真值表应有三个输入：A、B和C。

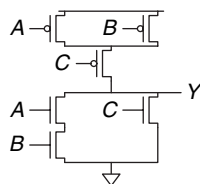


图1.52 神秘原理图

练习1.89 仅使用伪nMOS逻辑门实现以下三输入门。你的门接收三个输入： A 、 B 和 C 。使用最少数量的晶体管。

(a) 三输入或非门 (b) 三输入
与非门 (c) 三输入与门

练习1.90 *Resistor-Transistor Logic (RTL)* 使用nMOS晶体管将门输出拉低，并使用一个弱电阻在没有路径接地时将输出拉高。使用RTL构建的非门如图1.53所示。请绘制一个三输入RTL或非门，使用最少数量的晶体管。

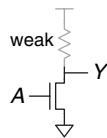


图1.53 RTL非门

面试问题

这些问题是数字设计岗位面试中常被问到的。

任务 离子1.1 绘制一个CMOS四输入与非门的晶体管级电路 或门。

问题1.2 国王收到了64枚金币作为税收，但有理由相信其中一枚是伪造的。他召你来找出假币。你有一个天平，两边可以放硬币。你需要使用天平多少次才能找到那枚较轻的假币？

问题1.3 教授、助教、数字设计学生和新生田径明星需要在漆黑的夜晚过一座摇晃晃的桥。桥非常不稳，每次只能有两个人同时过桥。他们只有一把手电筒，而桥太长了，无法将手电筒扔过去，所以必须有人把它带回来给其他人。新生田径明星过桥需要1分钟，数字设计学生过桥需要2分钟，助教过桥需要5分钟，教授

总是分心，过桥需要10分钟。所有人最快过桥需要多长时间？

